



(19) 대한민국특허청(KR)
(12) 등록특허공보(B1)

(45) 공고일자 2017년08월01일
(11) 등록번호 10-1763731
(24) 등록일자 2017년07월26일

(51) 국제특허분류(Int. Cl.)
G06F 9/38 (2006.01) G06F 13/00 (2006.01)
G06F 15/173 (2006.01) G06F 9/30 (2017.01)
G06F 9/46 (2006.01) H04L 12/933 (2013.01)
(52) CPC특허분류
G06F 9/3885 (2013.01)
G06F 13/00 (2013.01)
(21) 출원번호 10-2015-7028995
(22) 출원일자(국제) 2014년03월13일
심사청구일자 2015년10월13일
(85) 번역문제출일자 2015년10월13일
(65) 공개번호 10-2015-0132355
(43) 공개일자 2015년11월25일
(86) 국제출원번호 PCT/US2014/026427
(87) 국제공개번호 WO 2014/151773
국제공개일자 2014년09월25일
(30) 우선권주장
61/852,389 2013년03월15일 미국(US)
(56) 선행기술조사문헌
US20040202178 A1*
US20040213291 A1
US20060013207 A1
*는 심사관에 의하여 인용된 문헌

(73) 특허권자
인텔 코포레이션
미합중국 캘리포니아 95054 산타클라라 미션 칼리지 블러바드 2200
(72) 발명자
압달라, 모하마드
미국 95132 캘리포니아주 산 호세 선크레스트 애비뉴 3868
(74) 대리인
양영준, 백만기

전체 청구항 수 : 총 21 항

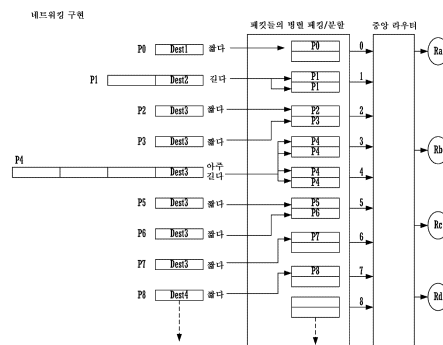
심사관 : 서광훈

(54) 발명의 명칭 회선 속도 상호 연결 구조를 구현하는 방법

(57) 요약

회선 속도 상호 연결 처리 방법이 개시된다. 이 방법은 입력 통신 경로로부터 초기 입력들을 수신하는 단계, 중간 입력들을 생성하기 위해 제1 스테이지 상호 연결 병렬 프로세서를 사용하여 초기 입력들의 사전 분류를 수행하는 단계, 및 결과 출력들을 생성하기 위해 제2 스테이지 상호 연결 병렬 프로세서를 사용하여 중간 입력들의 최종적인 결합 및 분할을 수행하는 단계를 포함한다. 이 방법은 제2 스테이지로부터 결과 출력들을 회선 속도로 전송하는 단계를 추가로 포함한다.

대표도



(52) CPC특허분류

G06F 15/17318 (2013.01)

G06F 9/3004 (2013.01)

G06F 9/3853 (2013.01)

G06F 9/462 (2013.01)

H04L 49/1523 (2013.01)

명세서

청구범위

청구항 1

회선 속도 상호 연결 처리(line speed interconnect processing) 방법으로서,

입력 통신 경로로부터 초기 입력들을 수신하는 단계와,

중간 입력들을 생성하기 위해 제1 스테이지 상호 연결 병렬 프로세서를 사용하여 상기 초기 입력들의 사전 분류(pre-sorting)를 수행하는 단계 - 상기 제1 스테이지 상호 연결 프로세서는 상기 초기 입력들 중 페어링(pairing)을 위해 검사될 후보들을 식별하기 위하여 상기 초기 입력들에 대해 사전 분류 및 사전 클러스터링(pre-clustering) 프로세스를 병렬로 수행하는 기능을 함 - 와,

결과 출력들을 생성하기 위해 제2 스테이지 상호 연결 병렬 프로세서를 사용하여 상기 중간 입력들의 최종적인 결합 및 분할을 수행하는 단계와,

상기 제2 스테이지로부터 상기 결과 출력들을 회선 속도로(at line speed) 전송하는 단계

를 포함하는

회선 속도 상호 연결 처리 방법.

청구항 2

삭제

청구항 3

제1항에 있어서,

상기 제2 스테이지 상호 연결 프로세서는 상기 결과 출력들을 회선 속도로 생성하기 위해 상기 중간 입력들의 위치 셔플링(shuffling), 페어링 및 분할을 병렬로 수행하는 기능을 하는

회선 속도 상호 연결 처리 방법.

청구항 4

제1항에 있어서,

상기 회선 속도 상호 연결 처리는 네트워킹 아키텍처에서 구현되고, 상기 초기 입력들은 네트워킹 패킷들을 포함하는

회선 속도 상호 연결 처리 방법.

청구항 5

제1항에 있어서,

상기 회선 속도 상호 연결 처리는 캐시 액세스 아키텍처에서 구현되고, 상기 초기 입력들은 캐시 라인들의 데이터에 대한 액세스 요청들을 포함하는

회선 속도 상호 연결 처리 방법.

청구항 6

제1항에 있어서,

상기 회선 속도 상호 연결 처리는 중재 아키텍처에서 구현되고, 상기 초기 입력들은 출력 대역폭을 이용하는 입력 스트림들을 포함하며, 상기 중재 아키텍처는 결과 출력 스트림들을 생성하기 위해 주파수 및 시간 다중화 중 적어도 하나를 병렬로 사용하여 상기 입력 스트림들 간에 중재하는

회선 속도 상호 연결 처리 방법.

청구항 7

제1항에 있어서,

상기 회선 속도 상호 연결 처리는 컴퓨터 명령어 아키텍처 디코더에서 구현되고, 상기 초기 입력들은 기계 명령어들로 병렬로 결합되거나 분할될 컴퓨터 명령어들을 포함하는

회선 속도 상호 연결 처리 방법.

청구항 8

제1항에 있어서,

상기 회선 속도 상호 연결 처리는 다이내믹 랜덤 액세스 메모리(dynamic random access memory; DRAM) 액세스 아키텍처에서 구현되고, 상기 초기 입력들은 DRAM 페이지들에서의 최적화된 결과 액세스들로 병렬로 페어링 또는 분할될 DRAM 페이지들에서의 액세스들을 포함하는

회선 속도 상호 연결 처리 방법.

청구항 9

컴퓨터 시스템에 의해 실행될 때, 상기 컴퓨터 시스템으로 하여금 회선 속도 상호 연결 처리 방법을 구현하게 하는 컴퓨터 판독 가능 코드를 갖는 비일시적 컴퓨터 판독 가능 메모리로서, 상기 방법은,

입력 통신 경로로부터 초기 입력들을 수신하는 단계와,

중간 입력들을 생성하기 위해 제1 스테이지 상호 연결 병렬 프로세서를 사용하여 상기 초기 입력들의 사전 분류를 수행하는 단계 - 상기 제1 스테이지 상호 연결 프로세서는 상기 초기 입력들 중 페어링을 위해 검사될 후보들을 식별하기 위하여 상기 초기 입력들에 대해 사전 분류 및 사전 클러스터링 프로세스를 병렬로 수행하는 기능을 함 - 와,

결과 출력들을 생성하기 위해 제2 스테이지 상호 연결 병렬 프로세서를 사용하여 상기 중간 입력들의 최종적인 결합 및 분할을 수행하는 단계와,

상기 제2 스테이지로부터 상기 결과 출력들을 회선 속도로 전송하는 단계

를 포함하는

컴퓨터 판독 가능 메모리.

청구항 10

삭제

청구항 11

제9항에 있어서,

상기 제2 스테이지 상호 연결 프로세서는 상기 결과 출력들을 회선 속도로 생성하기 위해 상기 중간 입력들의 위치 서플링, 페어링 및 분할을 병렬로 수행하는 기능을 하는

컴퓨터 판독 가능 메모리.

청구항 12

제9항에 있어서,

상기 회선 속도 상호 연결 처리는 네트워크 아키텍처에서 구현되고, 상기 초기 입력들은 네트워크 패킷들을 포함하는

컴퓨터 판독 가능 메모리.

청구항 13

제9항에 있어서,

상기 회선 속도 상호 연결 처리는 캐시 액세스 아키텍처에서 구현되고, 상기 초기 입력들은 캐시 라인들의 데이터에 대한 액세스 요청들을 포함하는

컴퓨터 판독 가능 메모리.

청구항 14

제9항에 있어서,

상기 회선 속도 상호 연결 처리는 중재 아키텍처에서 구현되고, 상기 초기 입력들은 출력 대역폭을 이용하는 입력 스트림들을 포함하며, 상기 중재 아키텍처는 결과 출력 스트림들을 생성하기 위해 주파수 및 시간 다중화 중 적어도 하나를 병렬로 사용하여 상기 입력 스트림들 간에 중재하는

컴퓨터 판독 가능 메모리.

청구항 15

제9항에 있어서,

상기 회선 속도 상호 연결 처리는 컴퓨터 명령어 아키텍처 디코더에서 구현되고, 상기 초기 입력들은 기계 명령어들로 병렬로 결합되거나 분할될 컴퓨터 명령어들을 포함하는

컴퓨터 판독 가능 메모리.

청구항 16

제9항에 있어서,

상기 회선 속도 상호 연결 처리는 다이나믹 랜덤 액세스 메모리(DRAM) 액세스 아키텍처에서 구현되고, 상기 초기 입력들은 DRAM 페이지들에의 최적화된 결과 액세스들로 병렬로 페어링 또는 분할될 DRAM 페이지들에의 액세스들을 포함하는

컴퓨터 판독 가능 메모리.

청구항 17

컴퓨터 시스템으로서,

시스템 메모리와,

상기 시스템 메모리에 결합된 중앙 프로세서 유닛

을 포함하되,

상기 중앙 프로세서 유닛은 컴퓨터 판독 가능 코드를 실행하여 상기 컴퓨터 시스템으로 하여금 회선 속도 상호 연결 처리 방법을 구현하게 하며, 상기 회선 속도 상호 연결 처리 방법은

입력 통신 경로로부터 초기 입력들을 수신하는 단계와,

중간 입력들을 생성하기 위해 제1 스테이지 상호 연결 병렬 프로세서를 사용하여 상기 초기 입력들의 사전 분류를 수행하는 단계 - 상기 제1 스테이지 상호 연결 프로세서는 상기 초기 입력들 중 페어링을 위해 검사될 후보들을 식별하기 위하여 상기 초기 입력들에 대해 사전 분류 및 사전 클러스터링 프로세스를 병렬로 수행하는 기능을 함 - 와,

결과 출력들을 생성하기 위해 제2 스테이지 상호 연결 병렬 프로세서를 사용하여 상기 중간 입력들의 최종적인 결합 및 분할을 수행하는 단계와,

상기 제2 스테이지로부터 상기 결과 출력들을 회선 속도로 전송하는 단계를 포함하는

컴퓨터 시스템.

청구항 18

삭제

청구항 19

제17항에 있어서,

상기 제2 스테이지 상호 연결 프로세서는 상기 결과 출력들을 회선 속도로 생성하기 위해 상기 중간 입력들의 위치 셔플링, 페어링 및 분할을 병렬로 수행하는 기능을 하는

컴퓨터 시스템.

청구항 20

제17항에 있어서,

상기 회선 속도 상호 연결 처리는 네트워킹 아키텍처에서 구현되고, 상기 초기 입력들은 네트워킹 패킷들을 포함하는

컴퓨터 시스템.

청구항 21

제17항에 있어서,

상기 회선 속도 상호 연결 처리는 캐시 액세스 아키텍처에서 구현되고, 상기 초기 입력들은 캐시 라인들의 데이터에 대한 액세스 요청들을 포함하는

컴퓨터 시스템.

청구항 22

제17항에 있어서,

상기 회선 속도 상호 연결 처리는 중재 아키텍처에서 구현되고, 상기 초기 입력들은 출력 대역폭을 이용하는 입력 스트림들을 포함하며, 상기 중재 아키텍처는 결과 출력 스트림들을 생성하기 위해 주파수 및 시간 다중화 중 적어도 하나를 병렬로 사용하여 상기 입력 스트림들 간에 중재하는

컴퓨터 시스템.

청구항 23

제17항에 있어서,

상기 회선 속도 상호 연결 처리는 컴퓨터 명령어 아키텍처 디코더에서 구현되고, 상기 초기 입력들은 기계 명령어들로 병렬로 결합되거나 분할될 컴퓨터 명령어들을 포함하는

컴퓨터 시스템.

청구항 24

제17항에 있어서,

상기 회선 속도 상호 연결 처리는 다이나믹 랜덤 액세스 메모리(DRAM) 액세스 아키텍처에서 구현되고, 상기 초기 입력들은 DRAM 페이지들에의 최적화된 결과 액세스들로 병렬로 페어링 또는 분할될 DRAM 페이지들에의 액세스들을 포함하는

컴퓨터 시스템.

발명의 설명

기술 분야

[0001] 본 출원은 2013년 3월 15일자로 출원된, 동시 계류 중이고 공동 양도된 발명의 명칭이 "A METHOD FOR IMPLEMENTING A LINE SPEED INTERCONNECT STRUCTURE"인 Mohammad A. Abdallah의 미국 가특허 출원 번호 제 61/852389호(본 명세서에 그 전체가 포함됨)에 기초하여 우선권을 주장한다.

[0002] 본 발명은 일반적으로 디지털 컴퓨터 시스템에 관한 것으로서, 보다 상세하게는, 명령어 시퀀스를 구성하는 명령어들을 선택하는 시스템 및 방법에 관한 것이다.

배경 기술

[0003] 의존적이거나 완전히 독립적인 다수의 작업들을 처리하는 데 프로세서가 필요하다. 이러한 프로세서의 내부 상태는 보통 프로그램 실행의 각각의 특정의 순간에 상이한 값들을 보유하고 있을 수 있는 레지스터들로 이루어져 있다. 프로그램 실행의 각각의 순간에, 내부 상태 이미지(internal state image)는 프로세서의 아키텍처 상태(architecture state)라고 불리운다.

[0004] 다른 함수(예컨대, 다른 스레드, 프로세스 또는 프로그램)를 실행하기 위해 코드 실행이 전환될 때, 새로운 함수가 그의 새로운 상태를 빌드(build)하기 위해 내부 레지스터들을 이용할 수 있도록 기계/프로세서의 상태가 저장되어야만 한다. 새로운 함수가 종료되면, 그의 상태가 폐기될 수 있고 이전의 컨텍스트의 상태가 복원될 것이며 실행이 재개된다. 이러한 전환 프로세스(switch process)는 컨텍스트 전환(context switch)이라고 불리우고, 특히 많은 수(예컨대, 64개, 128개, 256개)의 레지스터들 및/또는 비순차 실행(out of order execution)을 이용하는 최근의 아키텍처들에서 보통 수십 또는 수백 개의 사이클들을 포함한다.

[0005] 스레드 인식 하드웨어 아키텍처(thread-aware hardware architecture)에서, 하드웨어가 제한된 수의 하드웨어 지원 스레드들에 대해 다수의 컨텍스트 상태들을 지원하는 것이 보통이다. 이 경우에, 하드웨어는 각각의 지원된 스레드에 대한 모든 아키텍처 상태 요소들을 복제한다. 이것은 새로운 스레드를 실행할 때 컨텍스트 전환을

필요 없게 만든다. 그렇지만, 이것은 여전히 다수의 단점, 즉, 하드웨어로 지원되는 각각의 부가적인 스레드에 대한 모든 아키텍처 상태 요소들(즉, 레지스터들)을 복제하는 것의 면적, 능력(power) 및 복잡도를 가진다. 그에 부가하여, 소프트웨어 스레드들의 수가 명시적으로 지원되는 하드웨어 스레드들의 수를 초과하는 경우, 컨텍스트 전환이 여전히 수행되어야 한다.

[0006] 이것은 많은 수의 스레드를 요구하는 미세 입도 기반으로(on a fine granularity basis) 병렬 처리(parallelism)가 필요할 때 혼해진다. 중복 컨텍스트 상태 하드웨어 저장(duplicate context-state hardware storage)을 갖는 하드웨어 스레드 인식 아키텍처들은 비-스레드 방식 소프트웨어 코드(non-threaded software code)에는 도움을 주지 못하고 스레드 방식(threaded)으로 되어 있는 소프트웨어에 대한 컨텍스트 전환들의 수를 감소시킬 뿐이다. 그렇지만, 그 스레드들은 보통 대단위 병렬 처리(coarse grain parallelism)를 위해 구성되어 있으며, 그 결과 개시 및 동기화를 위한 소프트웨어 오버헤드가 과중하게 되고, 효율적인 스레딩 개시/자동 발생 없이, 함수 호출 및 루프 병렬 실행과 같은 소단위 병렬 처리(fine grain parallelism)를 남겨 두게 된다. 이러한 기술된 오버헤드들은 비-명시적으로/용이하게 병렬화되는/스레딩되는 소프트웨어 코드들에 대해 최신의 컴파일러 또는 사용자 병렬화 기법들을 사용한 이러한 코드들의 자동 병렬화의 어려움을 수반한다.

발명의 내용

[0007] 하나의 실시예에서, 본 발명은 회선 속도 상호 연결 처리 방법으로서 구현된다. 이 방법은 입력 통신 경로로부터 초기 입력들을 수신하는 단계, 중간 입력들을 생성하기 위해 제1 스테이지 상호 연결 병렬 프로세서를 사용하여 초기 입력들의 사전 분류(pre-sorting)를 수행하는 단계, 및 결과 출력들을 생성하기 위해 제2 스테이지 상호 연결 병렬 프로세서를 사용하여 중간 입력들의 최종적인 결합 및 분할을 수행하는 단계를 포함한다. 이 방법은 제2 스테이지로부터 결과 출력들을 회선 속도로 전송하는 단계를 추가로 포함한다.

[0008] 이상은 발명의 내용이고, 따라서 필요에 의해, 상세의 간략화, 일반화 및 생략을 포함하며; 그 결과 통상의 기술자라면 이 발명의 내용이 예시적인 것에 불과하고 결코 제한하기 위한 것이 아님을 잘 알 것이다. 청구항들에 의해서만 한정되는 본 발명의 다른 측면들, 발명 특징들 및 장점들이 이하에 기재되는 비제한적인 상세한 설명에서 명백하게 될 것이다.

도면의 간단한 설명

[0009] 본 발명은 첨부 도면들에서 제한으로서가 아니라 예로서 도시되어 있으며, 도면들에서 같은 참조 번호들은 유사한 요소들을 가리킨다.

도 1은 본 발명의 하나의 실시예에 따른, 회선 속도 상호 연결 구조의 개요를 나타낸 도면.

도 2는 본 발명의 하나의 실시예에 따른, 회선 속도 상호 연결 구조의 네트워킹 구현의 개요를 나타낸 도면.

도 3은 본 발명의 하나의 실시예에 따른, 회선 속도 상호 연결 구조의 프로세스들에 의해 사용되는 특정 용어를 나타낸 도면.

도 4는 본 발명의 하나의 실시예에 따른, FB 값 및 NP 값이 어떻게 사용되는지를 나타내는 제2 다이어그램.

도 5는 본 발명의 하나의 실시예에 따른, 초기 위치/패킷 번호, 새로운 패킷에서의 새 위치 번호, 및 결과 출력 패킷 번호 사이의 관계를 나타낸 다른 다이어그램.

도 6은 본 발명의 하나의 실시예에 따른, 앞서 기술된 모듈로 2 평가식(modulo two evaluating equation)의 동작의 보다 상세한 다이어그램.

도 7은 본 발명의 하나의 실시예에 따른, 캐시 액세스 구현의 동작을 나타낸 것으로서, 여기서 캐시 액세스들은 재배치되어 메모리 계층구조의 캐시 라인들의 크기들에 따라 정렬되어 있는 새로운 발신(outgoing) 캐시 액세스들에 위치됨.

도 8은 본 발명의 하나의 실시예에 따른, 재매핑 프로세스를 캐시의 포트들에 따라 나타낸 다이어그램.

도 9는 본 발명의 하나의 실시예에 따른, 2-스테이지 재매핑 프로세스를 캐시의 포트들에 따라 나타낸 다이어그램.

도 10은 본 발명의 하나의 실시예에 따른, 컴퓨터 명령어 구현의 동작을 나타낸 도면으로서, 여기서 컴퓨터 명령어들은 재배치되어 새로운 발신 컴퓨터 명령어들로 융합되거나 분할됨.

도 11은 본 발명의 하나의 실시예에 따른, 다수의 발신 목적지들에 대해 상이한 소스들이 중재되는 중재기(arbiter) 구현의 동작을 나타낸 도면.

도 12는 본 발명의 하나의 실시예에 따른, 다수의 발신 목적지들에 대해 상이한 소스들이 중재되는 중재기 구현의 다른 다이어그램.

도 13은 본 발명의 하나의 실시예에 따른, 다수의 발신 목적지들에 대해 상이한 소스들이 중재되는 중재기 구현의 다른 다이어그램.

도 14는 본 발명의 하나의 실시예에 따른, 모듈로 2 평가식들을 나타낸 스프레드시트의 그래픽 표현을 나타낸 도면.

도 15는 본 발명의 하나의 실시예에 따른, 누적 합 평가(cumulative sum evaluation) 프로세스의 동작을 나타낸 도면.

도 16은 본 발명의 하나의 실시예에 따른, 누적 합 평가를 병렬로 수행하는 회로를 나타낸 다이어그램.

도 17은 본 발명의 하나의 실시예에 따른, 누적 합의 평가를 수행하는 회로의 다이어그램.

도 18은 본 발명의 하나의 실시예에 따른, 누적 합의 평가를 수행하는 제2 회로의 다이어그램.

도 19는 병렬 가산기 구현에 대한 예시적인 아키텍처를 나타낸 도면.

도 20은 본 발명의 하나의 실시예에 따른, 병렬 캐리 저장 가산기(parallel carry save adder)를 나타낸 다이어그램.

도 21은 본 발명의 하나의 실시예에 따른, 스테이지 최적화된 고속 병렬 가산기의 일 실시예를 나타낸 도면.

발명을 실시하기 위한 구체적인 내용

- [0010] 본 발명이 하나의 실시예와 관련하여 기술되어 있지만, 본 발명이 본 명세서에 기재된 구체적인 형태들로 제한되는 것으로 의도되어 있지 않다. 그와 달리, 본 발명이 첨부된 청구항들에 의해 한정되는 본 발명의 범주 내에 타당하게 포함될 수 있는 이러한 대안들, 수정들 및 등가물들을 포함하는 것으로 의도되어 있다.
- [0011] 이하의 상세한 설명에서, 구체적인 방법 순서들, 구조들, 요소들 및 연결들과 같은 수많은 구체적인 상세들이 기재되어 있다. 그렇지만, 이 구체적인 상세들 및 다른 구체적인 상세들이 본 발명의 실시예들을 실시하는 데 이용될 필요는 없다는 것을 잘 알 것이다. 다른 상황에서, 본 설명을 불필요하게 모호하게 하는 것을 피하기 위해 공지된 구조들, 요소들, 또는 연결들이 생략되어 있거나, 특별히 상세히 기술되어 있지 않다.
- [0012] 본 명세서 내에서 "하나의 실시예" 또는 "일 실시예"에 대한 언급들은 그 실시예와 관련하여 기술된 특성의 특징, 구조 또는 특성이 본 발명의 적어도 하나의 실시예에 포함되어 있다는 것을 나타내기 위한 것이다. 본 명세서 내의 다양한 곳들에서 나오는 "하나의 실시예에서"라는 문구는 모두가 꼭 동일한 실시예를 말하는 것이 아니며 다른 실시예들과 상호 배타적인 별도의 또는 대안의 실시예들도 아니다. 더욱이, 일부 실시예들에서는 나타내고 있을 수 있지만 다른 실시예들에서는 그렇지 않을 수 있는 다양한 특징들이 기술되어 있다. 이와 유사하게, 일부 실시예들에 대해서는 요구사항들일 수 있지만 다른 실시예들에 대해서는 그렇지 않을 수 있는 다양한 요구사항들이 기술되어 있다.
- [0013] 이하의 상세한 설명의 어떤 부분들은 컴퓨터 메모리 내의 데이터 비트들에 대한 동작들의 절차들, 단계들, 논리 블록들, 처리 및 다른 심볼 표현들로 제시되어 있다. 이 설명들 및 표현들은 데이터 처리 분야의 통상의 기술자가 자신의 작업의 내용을 다른 통상의 기술자들에게 가장 효과적으로 전달하기 위해 사용되는 수단이다. 절차, 컴퓨터 실행 단계, 논리 블록, 프로세스 등은 여기에서 일반적으로 원하는 결과를 가져오는 자기 모순 없는 단계들 또는 명령어들의 시퀀스인 것으로 생각된다. 단계들은 물리적 양의 물리적 조작을 필요로 하는 것이다. 보통, 꼭 그렇지는 않지만, 이 양들은 컴퓨터 판독 가능 저장 매체의 전기 또는 자기 신호의 형태를 취하고, 컴퓨터 시스템에서 저장, 전송, 결합, 비교, 및 다른 방식으로 조작될 수 있다. 대체적으로 흔히 사용되기 때문에, 이 신호들을 비트, 값, 요소, 심볼, 문자, 용어, 숫자 등으로 지칭하는 것이 때로는 편리한 것으로 밝혀졌다.
- [0014] 그렇지만, 이 용어들 및 유사한 용어들 모두가 적절한 물리적 양들과 연관되어 있고 이 양들에 적용되는 편리한 라벨들에 불과하다는 것을 염두에 두어야 한다. 달리 구체적으로 언급하지 않는 한, 이하의 논의로부터 명백한 바와 같이, 본 발명 전체에 걸쳐 "처리" 또는 "엑세스" 또는 "기입" 또는 "저장" 또는 "복제" 등과 같은 용어를

이용한 논의가 컴퓨터 시스템의 레지스터 및 메모리 및 다른 컴퓨터 관독 가능 매체 내의 물리적(전자적) 양으로 표현된 데이터를 조작하고 컴퓨터 시스템 메모리 또는 레지스터 또는 다른 이러한 정보 저장, 전송 또는 디스플레이 디바이스 내의 물리적 양으로 유사하게 표현되는 다른 데이터로 변환하는 컴퓨터 시스템 또는 유사한 전자 컴퓨팅 디바이스의 동작 및 프로세스를 말한다는 것을 잘 알 것이다.

[0015] 본 발명의 실시예들은 극히 낮은 지연 시간을 요구하는 응용들에서 사용하기 위한 회선 속도 상호 연결 구조(line speed interconnect structure)를 구현한다. 많은 네트워킹이 지연 시간을 용납하지 않는 이러한 응용들에 영향을 미친다. 착신 패킷들을 상이한 목적지들로 포워딩하기 위해, 초고속 회로가 요구된다. 이하의 도면들은 이러한 고속 회로의 몇몇 실시예들을 예시하고, 패킷들을 교환하는 네트워킹 환경에서, 상이한 캐시 라인들에의 액세스들 및 상이한 열린 DRAM 페이지들에의 액세스들을 관리하는 처리 환경에서, 그리고 매크로 명령어들을 대응하는 병합 및/또는 분할된 마이크로명령어들로 병합 및/또는 분할하는 임의의 처리 환경에서 이러한 구조의 사용을 예시하고 있다.

[0016] 많은 구현들에서, 요소들은 병렬화된 요청들 또는 패킷들이고, 하나 또는 2개의 일반 동작들을 거친다. 예를 들어, 그들 중 2개 이상이 결합, 병합 또는 그룹화되어 통합된/균일한 요청 또는 패킷을 형성할 수 있다. 다른 동작은 요청 또는 패킷을 2개 이상의 요청들 또는 패킷들로 분할 또는 단편화(fragmenting)하는 것일 수 있다. 다른 예는 가변 길이 네트워킹 패킷들이다. 동일한 개념들이 고정 크기 패킷들에 적용될 수 있고, 여기서 다수의 고정 크기들이 지원된다.

[0017] 이하의 도면들은 이러한 회선 속도 상호 연결 구조의 상이한 실시예들을 예시하고 있다. 유의할 점은, "목적지"라는 용어가 꼭 최종 목적지를 의미하는 것은 아니라는 것이다.

[0018] 도 1은 본 발명의 하나의 실시예에 따른, 회선 속도 상호 연결 구조의 개요도를 나타낸 것이다. 도 1의 개요도는 회선 속도 상호 연결이 어떻게 2개의 스테이지들을 포함하는지를 나타내고 있다.

[0019] 도 1의 실시예에서, 제1 스테이지는 초기 입력들을 수신하고 초기 입력들에 대해 사전 분류(pre-sorting)/클러스터링(clustering)을 수행한다. 제1 스테이지의 출력들은 최종적인 재매핑(remapping)을 수행하는 제2 스테이지로 전달된다. 제2 스테이지로부터의 출력들은 이어서 회선 속도로 전달된다. 사전 분류 스테이지는, 예를 들어, 목적지 ID의 첫 번째로 일치하는 것(first match) 등을 사용하여, 페어링(pairing)을 위해 검사될 초기 입력들 중의 잠재적인 후보들을 식별하는 기능을 한다. 제2 스테이지는 이어서 위치 셔플링(position shuffling), 페어링(pairing) 또는 분할을 수행한다. 입력들을 결합 및 분할하여 결과 출력들을 병렬 방식으로 생성할 수 있는 것에 의해 회선 속도가 달성된다.

[0020] 도 1의 구조는 후속 도면들에서 기술될 다수의 상이한 구현들에서 적용될 수 있다. 구현들은 네트워킹 아키텍처, 캐시 액세스 아키텍처, DRAM 액세스 아키텍처, 및 중재 아키텍처, 그리고 컴퓨터 명령어 결합/분할 아키텍처를 포함한다. 이 구현들 전체에 걸쳐, 여기서 도 1에 나타낸 전체적인 구조는 입력들을 결과 출력들로 회선 속도 결합/분할하는 것을 제공한다.

[0021] 도 2는 본 발명의 하나의 실시예에 따른, 회선 속도 상호 연결 구조의 네트워킹 구현의 개요를 나타낸 것이다.

[0022] 앞서 기술된 바와 같이, 일련의 입력 요청들, 액세스들, 데이터, 명령어들 또는 패킷들이 그 입력들을 상이한 출력들로 재매핑/다중화 또는 셔플링하는 것을 거치는 많은 응용들이 있고; 그에 대한 아주 통상적인 이유는 그 입력들 중 2개 이상이 서로 병합, 결합 또는 그룹화되는 경우이고, 다른 이유는 그 입력들이 분할, 분리, 단편화 또는 브로드캐스트될 때이며, 이 2 가지 이유들은 입력들의 임의의 인터페이스의 출력들에의 완전한 재매핑, 데이터 또는 요청들 또는 액세스들의 상호 연결, 셔플링, 명령어 디코딩, 라우팅 또는 임의의 다중화를 야기할 수 있다. 또한, 이 응용들 및 구현들의 다수에서, 회선 속도/파이프라인 속도가 이러한 재매핑 기능들에 의해 유지되는 것이 중요하다.

[0023] 도 2는 회선 속도 상호 연결 구조의 낮은 지연 시간 품질을 이용하는 네트워킹 구현을 나타낸 것이며, 여기서 네트워킹 패킷들은 착신 패킷들[도 2의 좌측에 있는 패킷들(P0 내지 P8)로서 나타내어져 있음]을 받아서 이들을 발신 패킷들[예를 들어, 중앙 라우터에 들어가는 화살표들(0 내지 8)로서 나타내어져 있음]로 처리하기 위해 재매핑/결합/셔플링/분할 프로세스를 거친다. 중앙 라우터는 이어서 패킷들을 그들의 목적지들(Ra 내지 Rd로 나타내어져 있음)로 송신한다.

[0024] 이와 같이, 도 2는 착신 패킷들의 목적지들 및 크기들에 의해 그 패킷들이 어떻게 분할/결합/셔플링/재정렬되는지가 결정되는 프로세스를 나타내고 있다. 하나의 실시예에서, 목표는 [동일한 임시 경로(transient path)를 거쳐] 동일한 다음 목적지로 가는 중인 2개의 짧은 패킷들을 페어링하는 것이다. 페어링에 대한 이상적인 조건

은 양 패킷이 짧은 패킷이고 그 패킷들이 동일한 목적지로 향해 가는 경우일 것이다. 그에 부가하여, 하나의 실시예에서, 목표는 상이한 크기의 착신 패킷들을 받고 분할/결합/서플링/ 재정렬 프로세스를 수행하여 중앙 라우터로 송신되는 균일한 크기의 발신 패킷들을 생성하는 것이다. 각각의 발신 패킷은 다수의 짧은 패킷들이 어떻게 결합될 수 있는지를 보여주기 위해 상반부 및 하반부(예컨대, 짝수 또는 홀수)를 갖는 것으로 나타내어져 있다.

[0025] 도 2에 도시된 바와 같이, P0는 목적지 1로 가는 중이고 짧은 패킷이며, 발신 패킷 0에 위치된다. 다음 패킷 P1은 중간 크기의 패킷이며, P0과 상이한 목적지로 가는 중이다. P1은 발신 패킷 1의 상반부 및 하반부로 정렬된다. P2 및 P3 둘 다는 둘 다가 동일한 목적지(이 경우에, 목적지 3)로 가는 중인 짧은 패킷들이다. 그에 따라, P2 및 P3은 발신 패킷 2로 결합된다. P4는 큰 패킷이고, 큰 패킷이 2개 이상의 발신 패킷들로 분할되는 일례를 나타내고 있으며, 여기서 P4는 발신 패킷 3 및 4를 차지하도록 분할되는 것으로 나타내어져 있다. P5 및 P6 둘 다는 동일한 목적지로 가는 중인 짧은 패킷이고, 따라서 발신 패킷 5로 결합된다. P7 및 P8 둘 다는 상이한 목적지들로 가는 중인 짧은 패킷이고, 따라서 결합될 수 없다. 따라서 그들 각각은 그들 자신의 각자의 발신 패킷 6 및 7에 할당된다. 그리고 따라서 이 프로세스가 수신되는 착신 패킷들 모두에 대해 계속된다. 도 2에 나타난 바와 같이, 이 프로세스는 병렬로 그리고 아주 작은 지연 시간(즉, 회선 속도)으로 구현된다.

[0026] 도 3은 본 발명의 하나의 실시예에 따른, 회선 속도 상호 연결 구조의 프로세스들에 의해 사용되는 특정 용어를 나타낸 것이다. 이 실시예에서, 용어 FB=0은 짧은 패킷들을 나타내고, 용어 FB=1은 긴 패킷들을 나타내며, 용어 FB=3은 아주 긴 패킷들을 나타낸다. 용어 NP=0은 동일한 목적지를 가지는 패킷들을 나타낸다. 앞서 기술된 바와 같이, 하나의 실시예에서, 목표는 (동일한 임시 경로를 거쳐) 동일한 다음 목적지로 가는 중인 2개의 짧은 패킷들을 페어링하는 것이다. 페어링에 대한 이상적인 조건은 양 패킷이 짧은 패킷이고 그 패킷들이 동일한 목적지로 향해 가는 경우일 것이다. 이와 같이, 도 3에 나타난 표는 FB 값 및 NP 값이, 착신 패킷들이 자체적으로 발신 패킷들로 페어링, 분할 또는 송신될 수 있는지를 빠르게 평가하기 위해 어떻게 사용될 수 있는지를 나타내고 있다. 이러한 방식으로, FB는 버킷/블록/패킷이 채워져 있는지 여부를 나타내는 것으로 생각될 수 있고, NP는 그것들이 동일한 목적지로 향해 가는지를 나타낼 수 있다[예컨대, 점선 상자(301) 내에 쌍으로 나타내어져 있음]. 도 4에 나타난 "sdx"라는 용어는 목적지 "x"로 가는 중인 짧은 패킷을 지칭하고, "ldx"라는 용어는 목적지 "x"로 가는 중인 긴 패킷을 지칭한다.

[0027] 도 4는 본 발명의 하나의 실시예에 따른, FB 값 및 NP 값이 어떻게 사용되는지를 나타내는 제2 다이어그램을 도시한 것이다. 도 4는 모든 착신 패킷들을 처리하고 착신 패킷들의 발신 패킷들로의 분할/결합/재정렬을 결정하기 위해 평가식이 사용되는 방식을 나타내고 있다. 그에 부가하여, 이 평가는 병렬로 수행된다.

[0028] 앞서 기술된 바와 같이, 목표는 (예컨대, 동일한 임시 경로를 거쳐) 동일한 다음 목적지로 가는 중인 2개의 짧은 패킷들을 페어링하는 것이다. 양 패킷에 대한 조건은 FB=0이어야 하고(예컨대, 양 패킷이 짧은 패킷임), 나중의 패킷은 NP=0을 가져야 한다(예컨대, 나중의 패킷은 이전의 패킷과 동일한 목적지를 가지며, 따라서 페어링될 수 있음). 평가식은 $M5 = G5 + OR(MOD(G5, 2), 15) * (MOD(E5 + MOD(SUM(MS2:M4), 2), 2))$ 로서 쓰여질 수 있고, 여기서 MOD는 모듈로 2 산술(modulo two arithmetic)을 지칭한다. 이것은 요소 E5에 대한 새 위치를 평가하는 것에 대한 일례를 나타낸 상자(401)에 나타내어져 있다. 상자(402)는 요소 E6에 대한 새 위치를 평가하는 것에 대한 다른 예를 나타내고 있다. 6 위의 별표는 6이 분할의 첫 번째 부분의 위치라는 것을 나타낸다. 분할의 두 번째 부분의 위치는 분할의 첫 번째 부분의 위치에 2를 가산하는 것에 의해 로컬적으로 발생된다.

[0029] 상기 식에서의 OR는 NP 또는 FB가 설정되는지를 알아내기 위한 것이다. 어느 하나가 설정되어 있는 경우, 평가 프로세스는 버블(bubble)을 생성할 것이고, 버블은 상반부 또는 하반부(예컨대, 짝수 또는 홀수) 위치에 있을 것이다. 평가는 현재 위치와 현재 위치 이전의 버블들의 모든 누적들을 합산한다. 수식의 동작은 도 4에 도시된 표에 나타내어져 있고, 또한 도 4의 하반부에 나타내어져 있으며, 여기서 착신 패킷들(I0 내지 I7)은 그들이 발신 패킷들(0 내지 7)로 어떻게 처리되는지에 관해 나타내어져 있다.

[0030] 도 5는 본 발명의 하나의 실시예에 따른, 초기 위치/패킷 번호, 새로운 패킷에서의 새 위치 번호, 및 결과 출력 패킷 번호 사이의 관계를 나타낸 다른 다이어그램을 도시한 것이다. 착신 패킷은 "S"로 표시된 것과 같이 짧은 패킷일 수 있거나, "L"로 표시된 것과 같이 긴 패킷일 수 있거나, "VL"로 표시된 것과 같이 아주 긴 패킷일 수 있다. 이 예에서, 목적지들 모두는 용어 "D1"로 표시된 바와 같이 동일하다. 이와 같이, 도 5는 초기 패킷 위치들이 어떻게 결과 발신 패킷들의 새로운 패킷 위치들로 변경되는지의 예시이다. 어떤 패킷들은 결합되고(예컨대, 패킷 0 및 패킷 1), 어떤 패킷들은 분할된다(예컨대, 패킷 2 및 패킷 6). 상세하게는, 도 5는 아주 큰 패킷(예컨대, 패킷 6)이 어떻게 출력 패킷들을 차지하도록 분할되는지를 나타내고 있다. 초기 패킷들의 이 처리

는 앞서 기술된 모듈로 2 평가식에 따라 병렬로 수행된다.

- [0031] 도 6은 본 발명의 하나의 실시예에 따른, 앞서 기술된 모듈로 2 평가식의 동작의 보다 상세한 다이어그램을 나타낸 것이다.
- [0032] 앞서 기술된 바와 같이, 많은 구현들에서, 요소들은 병렬화된 요청들 또는 패킷들이고, 하나 또는 2개의 일반 동작들을 거친다. 그들 중 2개 이상이 결합, 병합 또는 그룹화되어 통합된/균일한 요청 또는 패킷을 형성할 수 있다. 다른 동작은 요청 또는 패킷을 2개 이상의 요청들 또는 패킷들로 분할 또는 단편화하는 것일 수 있다. 제1 예는 가변 길이 네트워킹 패킷들이다. 동일한 개념들이 고정 크기 패킷들에 적용될 수 있고, 여기서 다수의 고정 크기들이 지원된다. 그리고 그에 부가하여, 이 평가에서, 나중의 패킷은 $NP=0$ 을 가져야 한다(예컨대, 나중의 패킷은 이전의 패킷과 동일한 목적지를 가지며, 따라서 페어링될 수 있음).
- [0033] 많은 구현들에서, 요소들은 병렬화된 요청들 또는 패킷들이고, 하나 또는 2개의 일반 동작들을 거친다. 그들 중 2개 이상이 결합, 병합 또는 그룹화되어 통합된/균일한 요청 또는 패킷을 형성할 수 있다. 다른 동작은 요청 또는 패킷을 2개 이상의 요청들 또는 패킷들로 분할 또는 단편화하는 것일 수 있다. 제1 예는 가변 길이 네트워킹 패킷들이다. 동일한 개념들이 고정 크기 패킷들에 적용될 수 있고, 여기서 다수의 고정 크기들이 지원된다.
- [0034] 도 6의 실시예는 라우팅 아키텍처 구현의 동작을 기술하고, 여기서 가변 크기의 초기 패킷들은 재배치되어 새로운 균일한 크기의 발신 패킷들에 위치된다. 이 구현에서, 발신 패킷들은 짝수 위치 및 홀수 위치를 가진다. 가변 크기의 초기 패킷들은, 앞서 기술된 바와 같이, 모듈로 2 함수에 따라 짝수 위치 및 홀수 위치에 위치된다. 이 실시예에서, "NP" 표시자는, 초기 패킷들에 대해 0으로 설정되어 있는 경우, 이 2개의 패킷들이, 동일한 중간/임시 목적지로 향해 가고 있을 때, 서로 결합 또는 병합될 수 있다는 것을 나타낸다. "NP"가 1로 설정되어 있는 경우, 패킷들이 상이한 목적지들로 향해 가고 있기 때문에, 패킷들이 서로 결합 또는 병합될 수 없다. 3개 이상을 그룹화하기 위한 그룹화/결합이, 모듈로 함수를 그에 따라 조절하는 것에 의해, 용이하게 될 수 있다.
- [0035] 그렇지만, FB 표시자가 1로 설정될 때, 큰 패킷이 2개의 더 작은 패킷들로 단편화될 필요가 있다[패킷은 전체 블록/버킷(통합된 홀수/짝수 슬롯들)을 필요로 함]. FB가 1로 설정되는 다른 이유는 동일한 중간/임시 목적지를 공유한 최초 병합된 2개의 패킷들이 2개의 상이한 최종 목적지들을 갖는 2개의 상이한 패킷들로 분할될 필요가 있을 때이다. 패킷이 4개의 조각들로 분할될 필요가 있는 경우, FB는 3으로 설정될 수 있을 것이다.
- [0036] 좌측에 있는 도 6의 다이어그램은 초기 패킷 번호/위치, FB 및 NP 값들, 그리고 목적지를 나타내고 있다. 도 6의 우측에는, 위치들에 걸친 버블들, 및 새 위치들이 도시되어 있다. 수식은 먼저 버블들에 의해 야기되는 초기 위치로부터의 누적 변위(cumulative displacement)를 계산한다. 이어서, 누적 변위의 누적 합을 초기 위치에 가산하는 것에 의해 새 위치가 계산된다. 이 누적적인 어떤 특성은 점선 타원으로 나타내어져 있고, 각각의 후속 위치가 어떻게 모든 이전의 위치들의 누적 합에 의해 결정되는지를 나타내고 있다. 그에 부가하여, 점선 화살표들은 초기 위치 및 NP 값이 어떻게 평가식에 포함되는지를 나타내고 있다.
- [0037] 도 7은 본 발명의 하나의 실시예에 따른, 캐시 액세스 구현의 동작을 나타낸 것으로서, 여기서 캐시 액세스들은 재배치되어 메모리 계층구조의 캐시 라인들의 크기들에 따라 정렬되어 있는 새로운 발신 캐시 액세스들에 위치된다.
- [0038] 앞서 기술된 바와 같이, 많은 구현들에서, 요소들은 병렬화된 요청들이고, 하나 또는 2개의 처리 동작들을 거칠 수 있다. 하나의 경우에, 요청들 중 2개 이상이 결합, 병합 또는 그룹화되어 통합된/균일한 요청을 형성할 수 있다. 다른 경우는 요청을 2개 이상의 요청들로 분할 또는 단편화하는 것일 수 있다.
- [0039] 도 7의 실시예는 상이한 메모리 요청들이 다른 캐시 라인들에 액세스하려고 시도하는 다른 메모리 요청들의 존재 시에 동일한 캐시 라인에 액세스하는 메모리/캐시 아키텍처를 나타내고 있다. 이 경우에, 동일한 캐시 라인에 액세스하는 2개 이상의 요청들을 병합하는 것이 요망된다.
- [0040] 예를 들어, 동일한 캐시 라인에 대한 요청들이 병합됨으로써 그 캐시 라인에 대해 하나의 요청만이 나가고 그 캐시 라인이 복귀하도록 요청들이 재정렬될 수 있다. 동일한 캐시 라인의 다중 요청 및 다중 복귀는 없다. 비정렬된 메모리 요청(예컨대, 2개의 상이한 캐시 라인들에 걸쳐 있는 데이터에 대한 요청)은 분할의 일례이고, 여기서 그 요청은 비정렬된 요청된 데이터를 포함하는 2개의 상이한 캐시 라인들에 대한 2개의 요청들로 분할된다.

- [0041] 이 실시예에서, 동일한 캐시 라인에 액세스하고 있는 2개의 요청들이 동일한 통합된 요청(홀수-짝수 슬롯들)으로 결합된다. 초기 메모리 요청들은, 앞서 기술된 바와 같이, 모듈로 2 평가 함수(modulo two evaluation function)에 따라 짝수 위치 및 홀수 위치에 위치된다. 이 실시예에서, "NP" 표시자는, 초기 요청들에 대해 0으로 설정되어 있는 경우, 이 2개의 요청들이, 동일한 캐시 라인에 액세스하고 있을 때, 서로 결합 또는 병합될 수 있다는 것을 나타낸다. "NP"가 1로 설정되어 있는 경우, 요청들이 상이한 캐시 라인들에 액세스하고 있기 때문에, 요청들이 서로 결합 또는 병합될 수 없다. 2개 초과를 그룹화하기 위한 그룹화/결합이, 모듈로 함수를 그에 따라 조절하는 것에 의해, 용이하게 될 수 있다.
- [0042] 그렇지만, FB 표시자가 1로 설정될 때, 비정렬된 메모리 요청이 2개의 상이한 캐시 라인 액세스들로 분할될 필요가 있다. 액세스가 4개의 캐시 라인 액세스들[예를 들어, 2개 이상의 캐시 라인에 액세스하는 것을 필요로 하는 특수 메모리 요청들(예컨대, 문자열, 버퍼 복사, I/O 요청들 등)]로 분할될 필요가 있는 경우 FB가 3으로 설정될 수 있을 것이다. 앞서 기술된 바와 같이, 평가 수식은 먼저 버블들에 의해 야기되는 초기 위치로부터의 누적 변위를 계산한다. 이어서, 누적 변위의 누적 합을 초기 위치에 가산하는 것에 의해 새 위치가 계산된다.
- [0043] 유의할 점은, (네트워킹 경우와 상이한) 이 메모리/캐시 구현에서, 2개의 캐시 라인 요청들이 결합될 때, 그 요청들이 단지 단일의 요청으로 되고 블록/버킷의 홀수/짝수 슬롯을 물리적으로 차지하지 않는다는 것이다. 그러나, 홀수 및 짝수 슬롯은 캐시 라인 내의 2개의 상이한 주소들을 나타낸다.
- [0044] 유의할 점은, 이 구현에서, 각각의 홀수-짝수 슬롯들이 캐시 시스템/메모리 시스템으로의 상이한 포트 또는 버스로 독립적으로 전달될 수 있는 액세스를 나타낸다는 것이다.
- [0045] 그에 부가하여, 유의할 점은, 이 메모리/캐시 구현에서, 버블의 개념이 주어진 캐시 라인에 액세스하기 위해 짝수 및 홀수 슬롯들 둘 다를 이용할 수는 없는 경우라는 것이다.
- [0046] 다른 실시예에서, 도 7에 의해 기술되는 이 동일한 메모리 액세스 개념이 DRAM 제어기들에 적용될 수 있고, 이 경우 다수의 요청들이 메모리 시스템에서의 동일한 열린 DRAM 페이지에 액세스하기 위해 서로 결합될 수 있다. 이러한 실시예에서, DRAM 페이지들은 메모리 계층구조의 캐시 라인들과 유사한 것으로 생각된다. 이 유사성은 특히 DRAM 페이지들이 열려 있거나 닫혀 있는 방식에 적용된다. 이 구현에서, DRAM에 대한 요청들은, 동일한 DRAM 페이지에 대한 요청들이 함께 이동됨으로써 그 요청들이 DRAM 페이지가 열려 있는 동안 그 DRAM 페이지에 액세스할 수 있도록, 재정렬된다. DRAM에 대한 요청들은, 닫혀 있는 DRAM 페이지에 대한 액세스에 비해, 그 페이지가 열려 있을 때 훨씬 더 빠르다. 닫혀 있는 DRAM 페이지를 여는 데 일정량의 지연 시간이 걸린다. 이와 같이, 동일한 DRAM 뱅크/DRAM 페이지에 대한 다수의 액세스들의 이점을 달성하기 위해 요청들이 동일한 버스/포트/채널/DRAM 메모리 뱅크로 재정렬된다.
- [0047] 도 8은 본 발명의 하나의 실시예에 따른, 재매핑 프로세스를 캐시의 포트들에 따라 나타낸 다이어그램을 도시한 것이다. 도 8의 좌측은 일련의 초기 캐시 액세스 요청들("A" 내지 "F"로 예시되어 있음)을 나타내고 있다. 본 발명의 평가 프로세스는 초기 캐시 액세스 요청들에 대해 사전 클러스터링/필터링/서플링 프로세스를 수행한다. 도 8에 초기 위치로서 나타낸 Mrs.는 이어서 새 위치들로의 매핑을 거친다. 이 새 위치들은 캐시 메모리의 포트들 또는 뱅크들에 대응한다. 도 8은 앞서 기술된 동일한 구조가 액세스들이 결합되어야 하는지[예컨대, LD(A) 및 LD(B)]를 결정하기 위해 어떻게 NP 값들을 사용할 수 있는지 그리고 액세스들이 분할되어야 하는지[예컨대, 2개의 상이한 코트들/뱅크들에 액세스할 필요가 있는 LD(F)]를 결정하기 위해 어떻게 FB 값들을 사용할 수 있는지를 나타내고 있다. 2개 이상의 캐시 라인에 걸쳐 있는 데이터를 요청하는 정렬되지 않은 액세스들의 경우에 액세스들은 보통 분할된다.
- [0048] 유의할 점은, 하나의 실시예에서, 캐시 전체가 단지 하나의 통합된 다중-포트 뱅크인 경우 또는 캐시가 분할될 수 있는 경우(이 경우, 캐시 라인이 다수의 뱅크들에 걸쳐 나누어져 있음) 각각의 뱅크가 하나 또는 몇 개의 포트들을 가지도록(이러한 경우, 페어링된 액세스들이 캐시 라인의 동일한 뱅크에 대한 것인지가 검사됨) 포트들이 구현될 수 있다는 것이다.
- [0049] 도 9는 본 발명의 하나의 실시예에 따른, 2-스테이지 재매핑 프로세스를 캐시의 포트들에 따라 나타낸 다이어그램을 도시한 것이다. 예를 들어, 도 8이 단일의 스테이지만을 나타내고 있지만, 도 9는, 예시를 위해, 본 발명의 하나의 실시예에 따른, 2-스테이지 프로세스를 나타내고 있다. 도 1의 논의에서 앞서 기술된 바와 같이, 2-스테이지 프로세스의 경우, 제1 스테이지는 초기 입력들(이 경우에, 캐시에의 액세스들임)의 사전 클러스터링/천이/서플링/그룹화를 수행한다. 제2 스테이지는 캐시에의 액세스들의 최종적인 매핑을 수행한다.
- [0050] 도 9의 실시예에서, 동일한 캐시 라인으로 향해 가는 로드 요청들을 찾아내기 위해, 착신 로드 요청들의 그룹을

스캔하여 첫 번째로 일치하는 것을 찾아내기 위해 비교 로직이 사용된다. 예를 들어, LD(A)가 검사될 때, 비교 로직은 동일한 캐시 라인으로 향해 가는 LD(D)를 찾아낸다. 이 결과, LD(D)가 LD(A) 옆에 있도록 셔플링되거나 가상적으로 연관된다. 비교 로직이 LD(E)를 찾아낸 경우에 LD(B)에 대해 동일한 일이 행해진다. 이것은, 도시된 바와 같이, 비교하여 첫 번째로 일치하는 것을 찾아내는 것으로서 구현될 수 있다.

[0051] 로드 요청들이 셔플링되어 재정렬될 때, 제2 스테이지로 전달될 대응하는 로드 요청이 없는 경우 사전 클러스터링 스테이지에서의 그들의 초기 위치들 및 결과 위치들이 무효화될 수 있다. 사전 클러스터링 스테이지로부터 재매핑 스테이지로의 로드 요청들의 제시는 가상 요청(virtual request)이라고 지칭된다.

[0052] 동일한 캐시 라인에 액세스할 때(예컨대, NP=0), 재매핑의 결과는 복수의 병합된 로드 요청들을 포함한다. 재매핑의 다른 결과는 비정렬된 액세스가 2개의 캐시 라인 액세스들로 분할되는 비정렬된 액세스 분할(예컨대, FB=3)을 포함한다.

[0053] 재정렬된 가상 요청들이 제2 스테이지에 의해 수신되면, 그들은 (예컨대, NP 변수 또는 FB 변수에 따라) 통합되고 병합된 발신 로드 요청들에 재매핑되고, 이 경우 어떤 가상 요청들은 결합되었고 어떤 가상 요청들은 분할되었다.

[0054] 유의할 점은, 최종적인 매핑 스테이지의 구현이 이전의 도면들에서 앞서 기술된 예들과 동일하다는 것이다. 도시된 바와 같이, 유효하지 않은 가상 요청들이 자동으로 삭제되거나 다른 방식으로 필터링 제거된다. 유효하지 않은 요청들이 필터링 제거된 후에 NP 비트가 설정되고 검사된다는 것은 주목할 만한 가치가 있다.

[0055] 도 10은 본 발명의 하나의 실시예에 따른, 컴퓨터 명령어 구현의 동작을 나타낸 것으로서, 여기서 컴퓨터 명령어들은 재배치되어 새로운 발신 컴퓨터 명령어들로 융합되거나 분할된다. 도 10은 이상의 도면들에서의 구조 스크립트가 착신 명령어들(명령어 0 내지 명령어 8)이 새로운 발신 명령어들을 생성하기 위해 융합되거나 분할될 수 있는 컴퓨터 명령어 구현에서 어떻게 사용될 수 있는지를 나타내고 있다. 이것은 도 10에서 명령어들의 최종적인 명령어들/마이크로 연산들로의 병렬 융합 및 분할로 나타내어져 있다. 블록은 마이크로 연산들로의 연산 융합 또는 연산 분할과 같은 명령어들의 병렬 패킹/분할을 구현한다. 앞서 기술된 실시예들과 유사한 방식으로, NP 값은 어느 명령어들이 융합되는지를 결정할 수 있고, FB 값은 어느 명령어들이 분할되는지를 결정할 수 있다. 앞서 기술된 실시예들과 유사한 방식으로, 결과 명령어들은 상반부 및 하반부(예컨대, 또는 홀수/짝수)를 가진다.

[0056] 도 11은 본 발명의 하나의 실시예에 따른, 다수의 발신 목적지들에 대해 상이한 소스들이 중재되는 중재기 구현의 동작을 나타낸 것이다. 도 11은 이상의 실시예들에 기술된 구조가 다수의 목적지들로 가고 있는 상이한 소스들 간의 중재를 수행하는 데 사용되는 구현을 나타낸 것이다. 예를 들어, 다수의 소스들로부터의 어떤 요청들은 동일한 목적지로 결합될 수 있다. 어떤 다른 요청들은 다수의 목적지들로 브로드캐스트될 수 있다. 앞서 기술된 실시예들과 유사한 방식으로, NP 값은 어느 요청들이 결합되는지를 결정할 수 있고, FB 값은 어느 요청들이 분할되는지를 결정할 수 있다.

[0057] 예를 들어, 하나의 실시예에서, NP=0는 병합 동작(결합)을 가져온다. 이것은 페어링/웨이크업 통신(예컨대, 전원 끄기/슬립, 리셋)에서 종종 보게 된다. 일례는 프린터 또는 다른 유형의 주변 기기들일 것이다. FB=1은 분할 동작(예컨대, 2개 이상으로의 분할)을 가져온다. 이것은 브로드캐스트 통신에서 종종 보게 된다.

[0058] 도 11의 실시예에서, NP=0는 병합 동작(결합)을 가져온다. 이것은 페어링/웨이크업 통신에서 종종 보게 되며, 일례는 프린터 또는 다른 유형의 주변 기기들에 대한 요청들일 것이다. 이 실시예에서, FB=1은 분할 동작(2개 이상으로의 분할)을 가져온다. 이것은 브로드캐스트 통신 또는 전원 끄기/슬립, 리셋에서 종종 보게 된다.

[0059] 도 12는 본 발명의 하나의 실시예에 따른, 다수의 발신 목적지들에 대해 상이한 소스들이 중재되는 중재기 구현의 다른 다이어그램을 도시한다. 도 12의 다이어그램은 명시적 2-스테이지 구현을 나타내고 있다. 제1 스테이지는 앞서 기술된 방식으로 사전 분류 및 사전 클러스터링을 수행한다. 제2 스테이지는 앞서 기술된 방식으로 분할에서 최종적인 분류 결합을 수행한다. 도 12의 실시예에서, 제1 스테이지와 제2 스테이지 사이에 도시된 입력은 상이한 대역폭 요구사항들을 가지는 브로드캐스트 스트림들이다. 이 브로드캐스트 스트림들은, 예를 들어, 도시된 바와 같이, 더 빠른 시간 다중화 또는 주파수 다중화 출력을 구현하는 단일의 더 높은 속도의 출력 브로드캐스트로 결합될 수 있다.

[0060] 도 13은 본 발명의 하나의 실시예에 따른, 다수의 발신 목적지들에 대해 상이한 소스들이 중재되는 중재기 구현의 다른 다이어그램을 도시한다. 도 13의 다이어그램에서, 출력들은 상이한 더 빠른 인터페이스들 또는 버스들 또는 채널들로서 도시되어 있다. 도 13의 좌측에 있는 입력들은 더 낮은 주파수/더 낮은 대역폭의 입력들을 포

합할 수 있다. 중재기는 도식된 바와 같이 이 입력들을 더 높은 주파수의 출력들을 갖는 더 높은 성과를 내는 것(higher-paying)으로 결합하거나 분할하는 기능을 한다. 출력들은, 예를 들어, 시간 다중화 또는 주파수 다중화를 사용하여 더 높은 대역폭을 가질 수 있다. 입력들은 출력 대역폭을 이용하는 스트림들을 포함한다. 중재기는 결과 출력 스트림들을 생성하기 위해 주파수 및/또는 시간 다중화를 사용하여 입력 스트림들 간에 중재한다. 페어링은 요청들이 최적의 DRAM 액세스 시간을 갖도록 요청들이 백투백으로(back to back) 정렬된다는 것을 암시한다. 도 13의 우측은 상이한 목적지들을 나타내고 있다. 예를 들어, 어떤 목적지들은 많은 상이한 최종 목적지들로 가는 중인 방송국들이다. 다른 목적지들은 단일 지점의 목적지들이다.

[0061] 도 14는 본 발명의 하나의 실시예에 따른, 모듈로 2 평가식들을 나타낸 스프레드시트의 그래픽 표현을 나타낸 것이다. 스프레드시트의 상부 부분은 직렬 모드 평가 프로세스(예컨대, 직렬 모드 FB/NP)를 나타낸다. 스프레드시트의 하부 부분은 병렬 모드 평가 프로세스(예컨대, 병렬 모드 FB/NP)를 나타낸다. 유의할 점은, 스프레드시트가 도 6에 기술된 평가 프로세스 동안 설명된 수식들을 나타내고 있다는 것이다. 수식들은 먼저 버블들에 의해 야기되는 초기 위치로부터의 누적 변위를 계산한다. 이어서, 누적 변위의 누적 합을 초기 위치에 가산하는 것에 의해 새 위치가 계산된다. 도 14의 상부 부분에 나타낸 바와 같은 직렬 프로세스에서, 이 누적 합은 사이클 단위 프로세스(a cycle by cycle process)에서 행해진다. 도 6의 설명에서 기술된 바와 같은 병렬 프로세스에서, 누적 합은 도 14의 하부 부분에 나타낸 바와 같이 병렬로 계산된다.

[0062] 도 15는 본 발명의 하나의 실시예에 따른, 누적 합 평가 프로세스의 동작을 나타낸 것이다. 도 15는 재귀적 항들을 펼치고(unroll) 중복 항들을 제로화(zero out)하는 것에 의해 모듈로 2 항 수식(modulo two term equation)들이 어떻게 단순화될 수 있는지의 일례를 나타낸 것이다. 이것은 모듈로 2 산술의 속성들을 이용한다. 도 14의 처음 3개의 라인들은 항들 M2, M3, 및 M4를 나타낸다. 이 항들은 도 14에 더 상세히 나타내어져 있다. 도 14로부터 복사되는 모듈로 항들은 여기서 도 15에 나타내어져 있다. 모듈로 항들이 펼쳐질 때, 중복 항들이 제로화될 수 있다. 이것은 도 15의 라인 4에 나타내어져 있으며, 여기서 M4의 중복 항이 제로화된다. 이 속성의 결과, 재귀적으로 합산될 필요가 있는 항들의 수가 감소되고, 따라서 수식을 구현하는 회로의 병렬화를 단순화시킨다.

[0063] 도 16은 본 발명의 하나의 실시예에 따른, 누적 합 평가를 병렬로 수행하는 회로를 나타낸 다이어그램을 도시한 것이다. 도 16은 곱셈 항들이 AND 게이트들을 사용하여 어떻게 구현되는지 및 덧셈 항들이 XOR 게이트들을 사용하여 어떻게 구현되는지를 나타내고 있다. 이와 같이, 도 16은 모듈로 2 덧셈 및 곱셈 둘 다를 수행하는 논리 회로를 나타내고 있다.

[0064] 이 다이어그램은 단일 클럭 사이클로 나타낸 모듈로 2 함수를 풀기 위해 사용되는 스테이지 최적화된 고속 재매핑 로직 구조를 나타내고 있다. 재매핑 로직은 복수의 1-비트 입력들이 1-비트 또는 2-비트 출력을 생성하기 위해 신속하게 가산될 필요가 있는 임의의 응용에 특히 적합하다. 재매핑 로직 구조의 목적은 맞닥뜨린(encountered) 패킷들의 매핑 또는 패킹 또는 분할을 병렬로 그리고 회선 속도로 행하는 것이다. 이것을 하기 위해, 이 구조는 2 가지 관찰 사항들을 이용한다. 첫 번째 관찰 사항은 도면에 도시된 바와 같이, 재귀 합 수식을 펼칠 때, 모듈로 2 산술에 따라 중복 합산 요소들이 제로화된다는 것이다. 두 번째 관찰 사항은 도면에 도시된 재귀 합 수식 내의 덧셈 요소들 및 곱셈 요소들이 어떻게 모듈로 2 산술에 따라 거동하는지이다. 이러한 조건들 하에서, 덧셈 요소들은 XOR 함수들로 되고, 곱셈 요소들은 AND 함수들로 된다. 이것은 재귀 합 수식이, 도면에 도시된 바와 같이, 논리 회로들에 매핑될 수 있게 한다. 괄호 안의 요소들은 곱셈이고, 따라서 AND 게이트들에 의해 처리된다. 괄호 밖의 요소들은 덧셈이고, 따라서 XOR 게이트들에 의해 처리된다. 수식은 직렬화되는 대신에 이제 공간에 완전히 펼쳐져 있다.

[0065] 재매핑의 새 위치들을 찾아내기 위해, 이 논리 함수들의 출력들이 병렬 가산기에 피드하는 데 사용되고, 이 병렬 가산기는 가상 요청들의 새 위치들로의 최종적인 재매핑을 생성한다. 이러한 방식으로, 회로는 앞서 기술된 2개의 동작들을 수행하고, 여기서 그 입력들(예컨대, 가상 요청들) 중 2개 이상이 결합, 병합 또는 그룹화되어 병합된 요청을 형성할 수 있다. 다른 동작은 가상 요청 또는 패킷을 2개 이상의 요청들로 분할 또는 단편화하는 것일 수 있다.

[0066] 이러한 방식으로, 회로는 직렬 덧셈 프로세스를 취하고, 여기서 각각의 가상 요청의 배치는 각각의 이전의 가상 요소의 배치에 의존하고 모듈로 2 함수를 병렬로 구현하여, 재매핑된 출력 요청들을 생성한다.

[0067] 도 17은 본 발명의 하나의 실시예에 따른, 누적 합의 평가를 수행하는 회로의 다이어그램을 도시한 것이다. 도 17은 평가 수식의 항들이 평가 회로의 하드웨어 구성요소들에 의해 어떻게 처리되는지를 나타내고 있다. 이 다이어그램은 단일 클럭 사이클로 나타낸 모듈로 2 함수를 풀기 위해 사용되는 스테이지 최적화된 고속 가산기 회

로[예컨대, 병렬 가산기(1700)]를 나타내고 있다. 회로는 복수의 1-비트 입력들이 1-비트 또는 2-비트 출력을 생성하기 위해 신속하게 가산될 필요가 있는 임의의 응용에 특히 적합하다. 병렬 가산기(1700)의 상세들은 이하에서 도 21에 추가로 기술된다.

[0068] 앞서 기술된 바와 같이, 회선 속도는 지연 시간을 용납하지 않는 이러한 응용들에서 큰 영향을 미친다. 하나의 예는 착신 패킷들을 상이한 목적지들로 포워딩해야만 하고, 따라서 초고속 회로가 필요한 네트워킹 응용들로부터의 것이다. 이상의 도면들은 이러한 고속 회로의 몇몇 실시예들을 예시하고, 패킷들을 교환/라우팅하는 네트워킹 환경에서, 상이한 캐시 라인들에의 액세스들 및 상이한 열린 DRAM 페이지들에의 액세스들을 관리하는 처리 환경에서, 그리고 매크로 명령어들을 대응하는 병합 및/또는 분할된 마이크로명령어들로 병합/융합 및/또는 분할/디코딩하는 임의의 처리 환경에서 이러한 구조의 사용을 예시하고 있다.

[0069] 그에 부가하여, 앞서 기술된 바와 같이, 많은 구현들에서, 입력 요소들은 병렬화된 요청들 또는 패킷들이고, 하나 또는 2개의 일반 동작들을 거친다. 그 입력들 중 2개 이상이 결합, 병합 또는 그룹화되어 통합된/균일한 요청 또는 패킷을 형성할 수 있다. 다른 동작은 요청 또는 패킷을 2개 이상의 요청들 또는 패킷들로 분할 또는 단편화하는 것일 수 있다. 제1 예는 가변 길이 네트워킹 패킷들이다. 동일한 개념들이 고정 크기 패킷들에 적용될 수 있고, 여기서 다수의 고정 크기들이 지원된다.

[0070] 이러한 기능을 수행하는 도 17의 회로는 직렬 모듈 합(serial modular sum)으로서 구성될 수 있지만(예컨대, 여기서, 2개의 요소들을 병합하는 것 및 2개 또는 4개로의 분할의 예들이 주어져 있기 때문에, 이는 모듈로 2 합수로 예시되어 있음), 유의할 점은, 이 기능이 또한 3개 이상을 다단계로 병합하기 위해 계층적 방식으로 적용될 수 있다는 것이다. 이러한 방식으로, 도 17은 회로 구성요소들 중 다수가 회선 속도를 달성하기 위해 그들과 연관된 병렬 가산기들을 갖는 병렬 AND-XOR 게이트들로 변환되는 신규의 병렬화 감소 방식을 나타낸 것이다.

[0071] 도 18은 본 발명의 하나의 실시예에 따른, 누적 합의 평가를 수행하는 제2 회로의 다이어그램을 도시한 것이다. 도 18이 도 17과 유사하지만, 도 18은 병렬 1-비트 가산기와 함께 사용되는 2-비트 가산기를 나타내고 있다. 도 18의 다이어그램은 초기 위치들을 받고 그로부터 최종 위치들을 계산하기 위한 수식들이 어떻게 병렬로 계산되는지를 나타내고 있다. 앞서 기술된 바와 같이, 수식의 요소들은 곱셈 항들과 덧셈 항들로 분해될 수 있다. 곱셈 항들은, 도시된 바와 같이, AND 게이트들에 의해 해결된다. 덧셈 항들은, 도시된 바와 같이, XOR 게이트들에 의해 해결된다. 도 18은 수식을 초기 위치 5[예컨대, $n(5)$]에 대해 푸는 회로의 일례를 나타낸 것이다. 초기 위치들(예컨대, 이 예에서, 초기 위치 1 내지 10) 각각은 도 18에 도시된 것과 유사한 대응하는 회로를 갖는 대응하는 수식을 가질 것이다. 이 수식들 각각의 결과들은 가산기(1700)와 같은 고속 병렬 가산기에의 입력들이다.

[0072] 도 19는 병렬 가산기 구현에 대한 예시적인 아키텍처를 나타낸 것이다. 도 19는 병렬 가산기를 구현하기 위해 4:2 캐리 저장 가산기가 어떻게 사용될 수 있는지를 나타내고 있다.

[0073] 도 20은 본 발명의 하나의 실시예에 따른, 병렬 캐리 저장 가산기들을 나타낸 다이어그램을 도시한 것이다. 도 20에 도시된 바와 같이, 종래의 병렬 4:2 캐리 저장 가산기들은 디스패치하기 위한 명령어들을 선택하는 데 또는 디스패치하기 위한 선택된 명령어들을 대응하는 디스패치 포트에 매핑하는 데 사용되는 프로세서 스케줄링에서의 준비 완료 비트(ready bit)들과 같은, 병렬 입력들의 합산을 수행하는 데 사용되거나, 예를 들어, 유효한 최소 엔트리들을 카운트하고 할당 기입 포트들을 그들에 할당하기 위해 유효 비트들을 합산하는 데 사용된다.

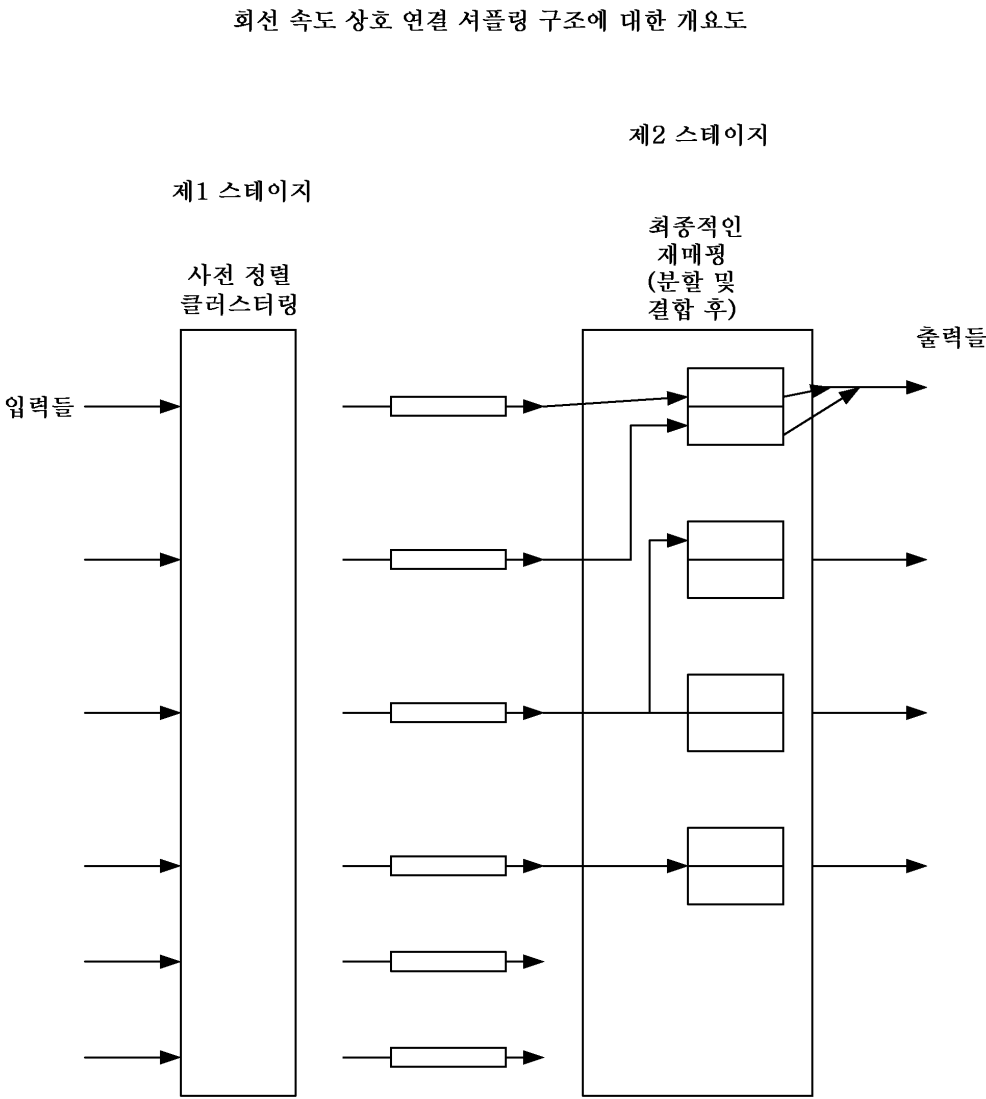
[0074] 도 21은 본 발명의 하나의 실시예에 따른, 스테이지 최적화된 고속 병렬 가산기의 일 실시예를 나타낸 것이다. 도 21의 실시예는 도 17에 도시된 병렬 가산기(1700)를 구현하는 데 사용될 수 있다. 도 21의 상부 부분은 입력들의 도착 시각을 나타내고, 또한 3개의 4:1 가산기들 및 하나의 2:1 가산기를 나타내고 있다. 도 21의 하부 부분은 4:1 가산기가 어떻게 구현되는지를 나타내고 있다. 도 21의 중간 부분은 그 위쪽에 있는 3개의 4:1 가산기들 및 하나의 2:1 가산기의 출력들을 수신하기 위해 결합된 2개의 3:2 캐시 저장 가산기들의 사용을 나타내고 있다. 이 구성은 종래의 병렬 가산기들과 비교하여 하나의 4:2 캐리 저장 가산기 스테이지를 절감하고, 따라서 도 21의 실시예를 훨씬 더 빠르게 만든다. 이 구현은 병렬 입력들 각각이 1-비트 입력들이고 회로가 그에 따라 최적화될 수 있다는 사실을 이용한다.

[0075] 도 21에서, 가산기가 디스패치하기 위한 명령어들을 선택하는 데 또는 디스패치하기 위한 선택된 명령어들을 대응하는 디스패치 포트에 매핑하는 데 사용되는 프로세서 스케줄링에서의 준비 완료 비트들과 같은, 1-비트 입력들의 합산을 수행하는 데 사용되거나, 예를 들어, 유효한 최소 엔트리들을 카운트하고 할당 기입 포트들을 그들에 할당하기 위해 유효 비트들을 합산하는 데 사용될 수 있다.

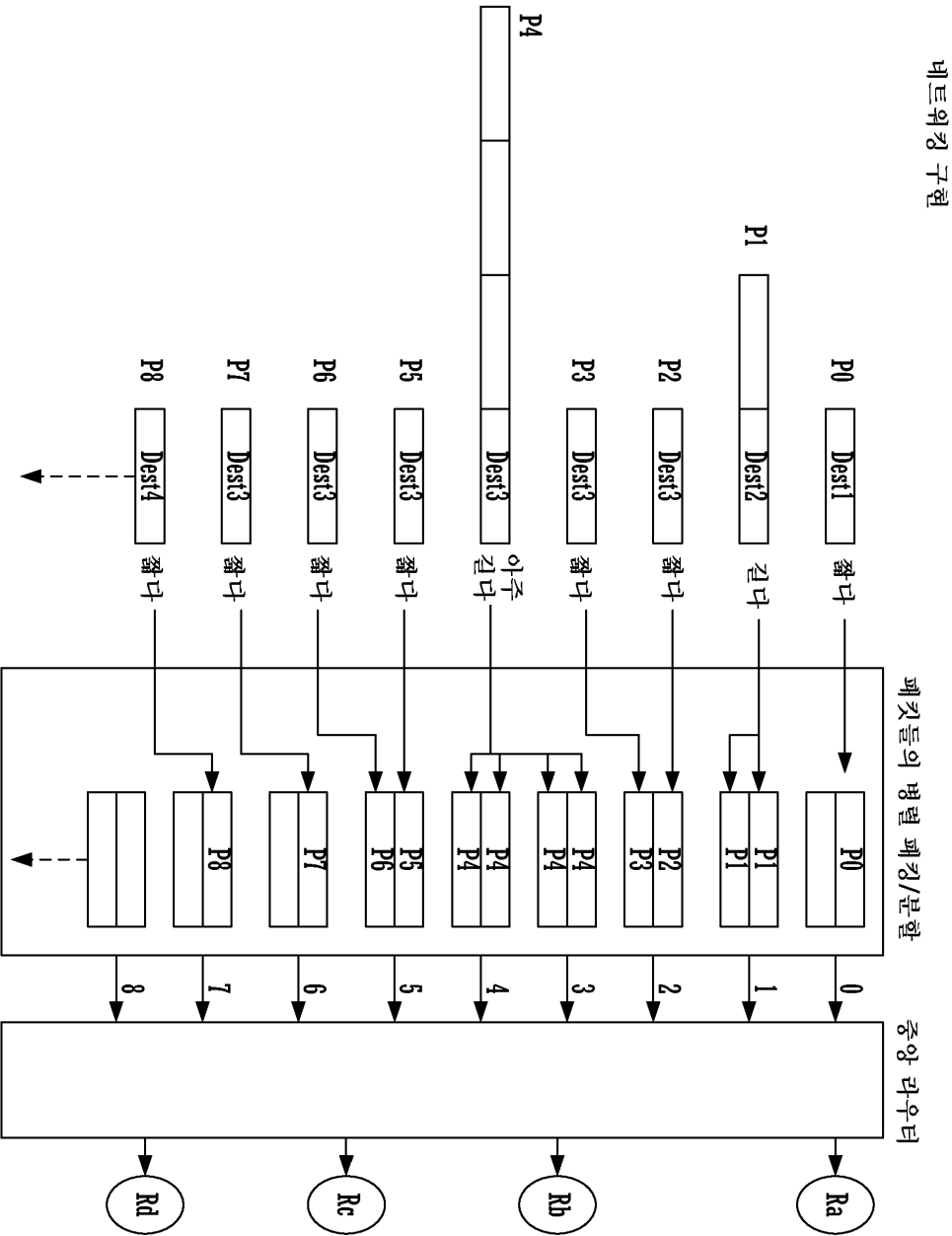
[0076] 이상의 기재는, 설명을 위해, 구체적인 실시예들을 참조하여 기술되어 있다. 그렇지만, 이상의 예시된 논의는 전수적인 것으로 의도되어 있지도 않고 본 발명을 개시된 정확한 형태들로 제한하는 것으로도 의도되어 있지 않다. 이상의 개시 내용을 고려하여 많은 수정들 및 변형들이 가능하다. 본 발명의 원리들 및 그의 실제 응용들을 가장 잘 설명함으로써 다른 통상의 기술자들이 본 발명 및 생각되는 특정의 용도에 적합할 수 있는 다양한 수정들을 갖는 다양한 실시예들을 가장 잘 이용할 수 있게 해주기 위해 실시예들이 선택되어 기술되었다.

도면

도면1



도면2

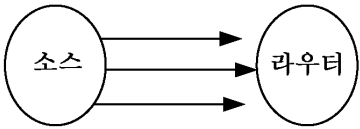


도면3

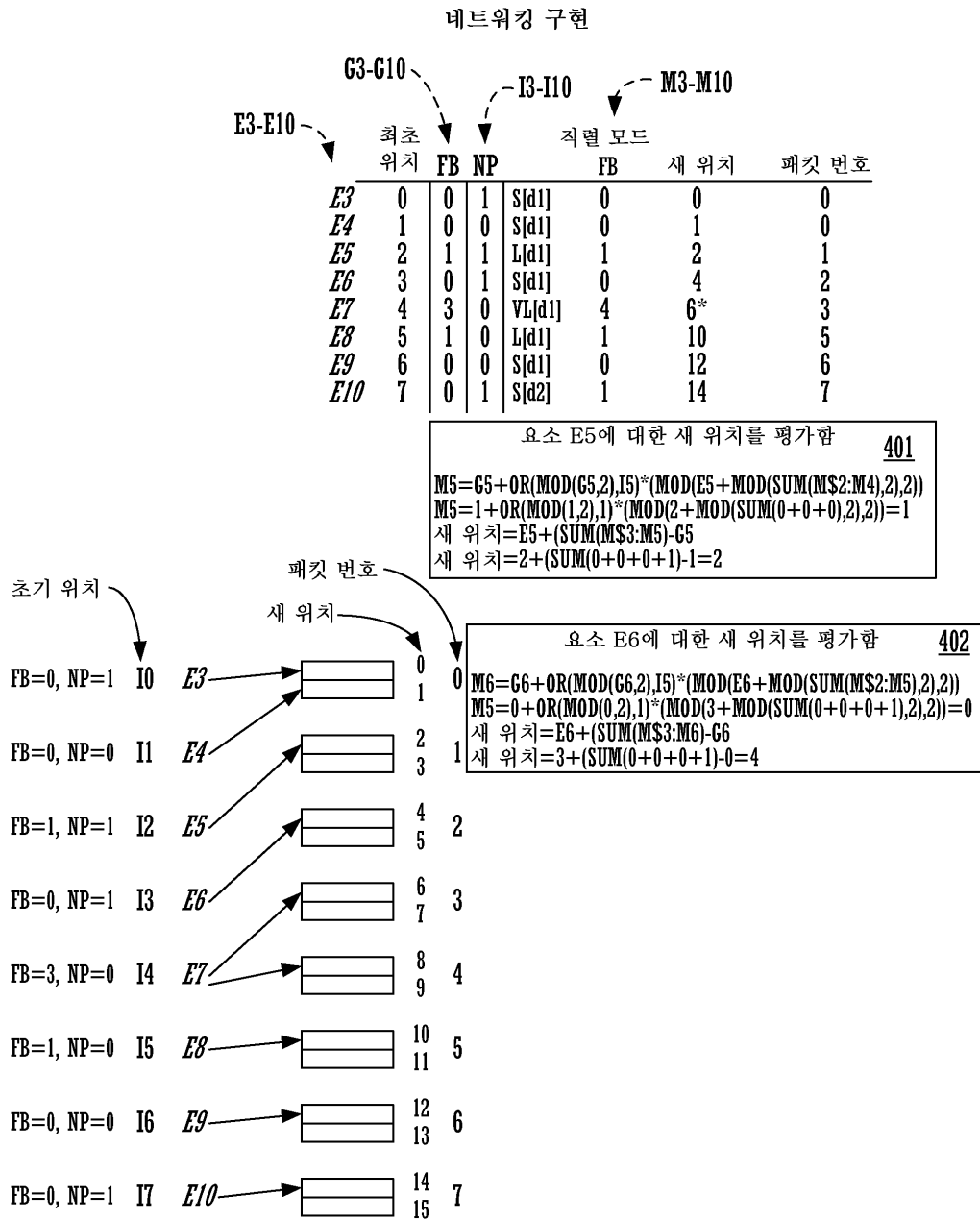
네트워킹 구현

짧은 패킷들에 대한 FB=0
긴 패킷들에 대한 FB=1
아주 긴 패킷들에 대한 FB=3
동일한 목적지에 대한 NP=0

FB	NP	
0	1	SD1
0	1	SD2
1	0	LDX
0	1	SD1
1	0	LDX
1	0	LDX
0	1	SD1
0	1	SD2
0	0	SD2
0	0	SD2
0	0	LDX

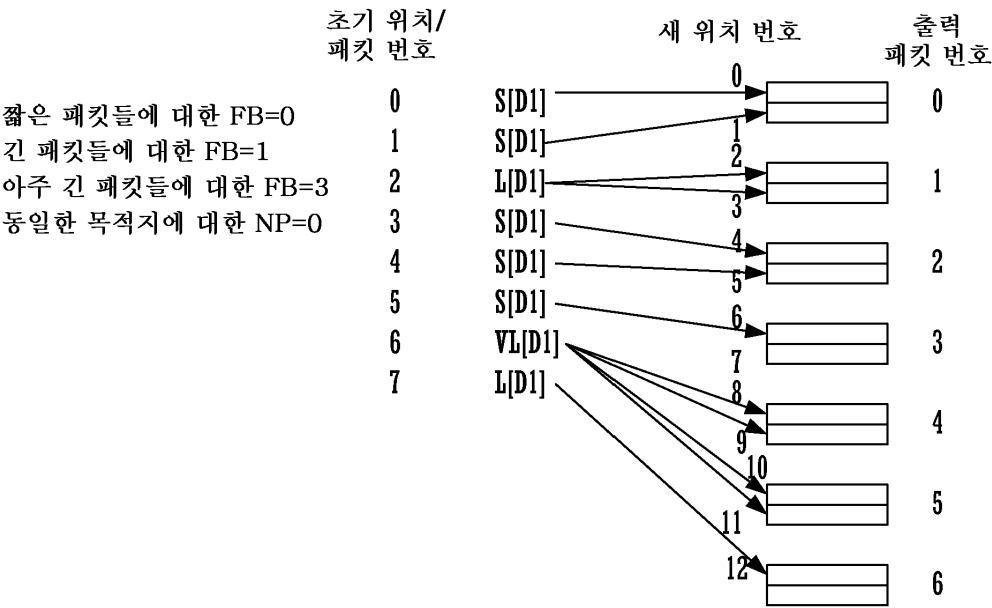


도면4

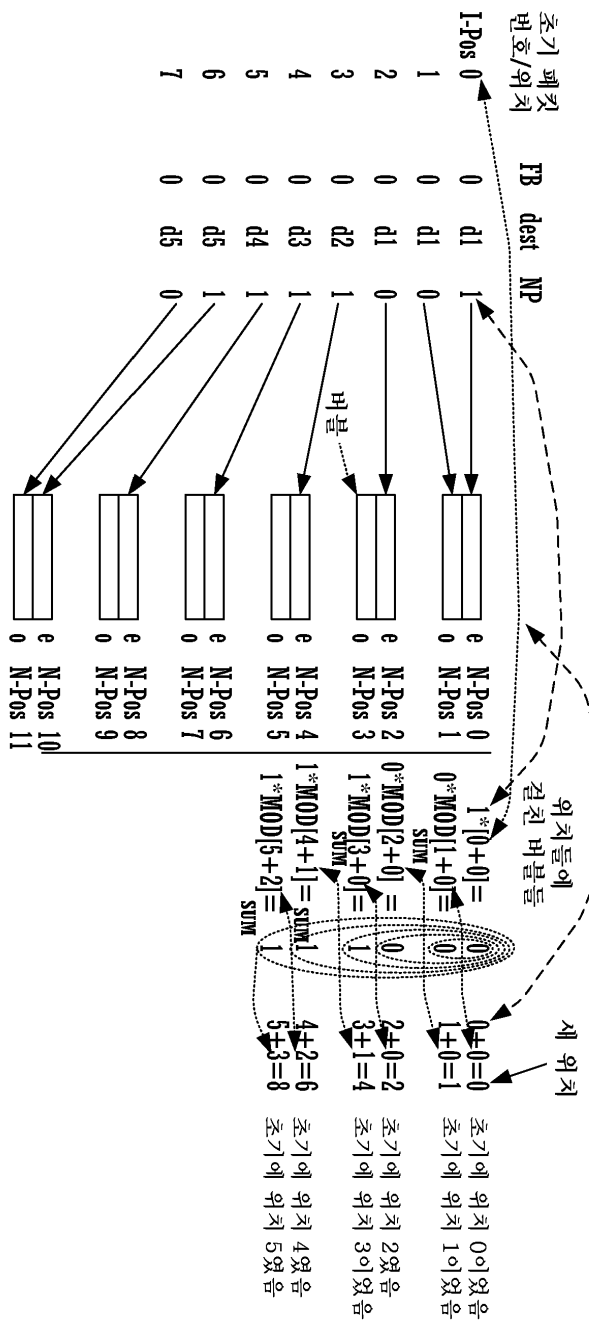


도면5

네트워킹 구현

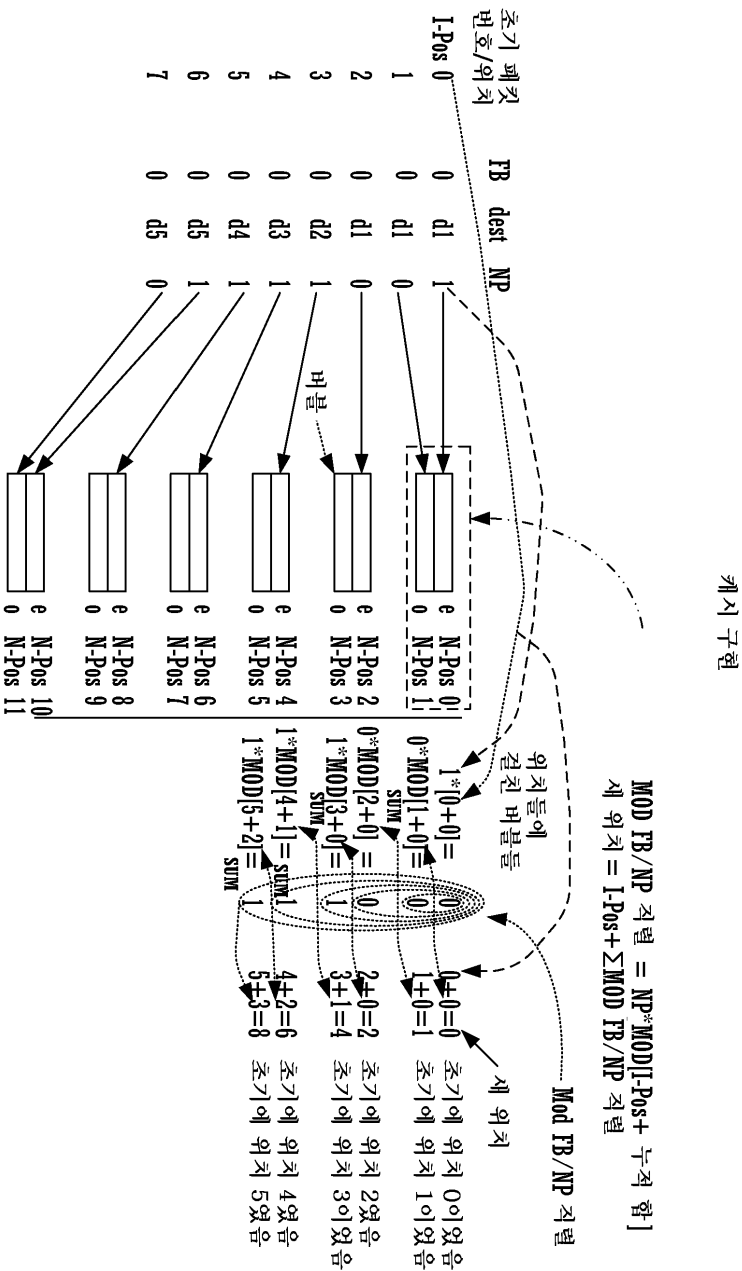


네트워킹 구현

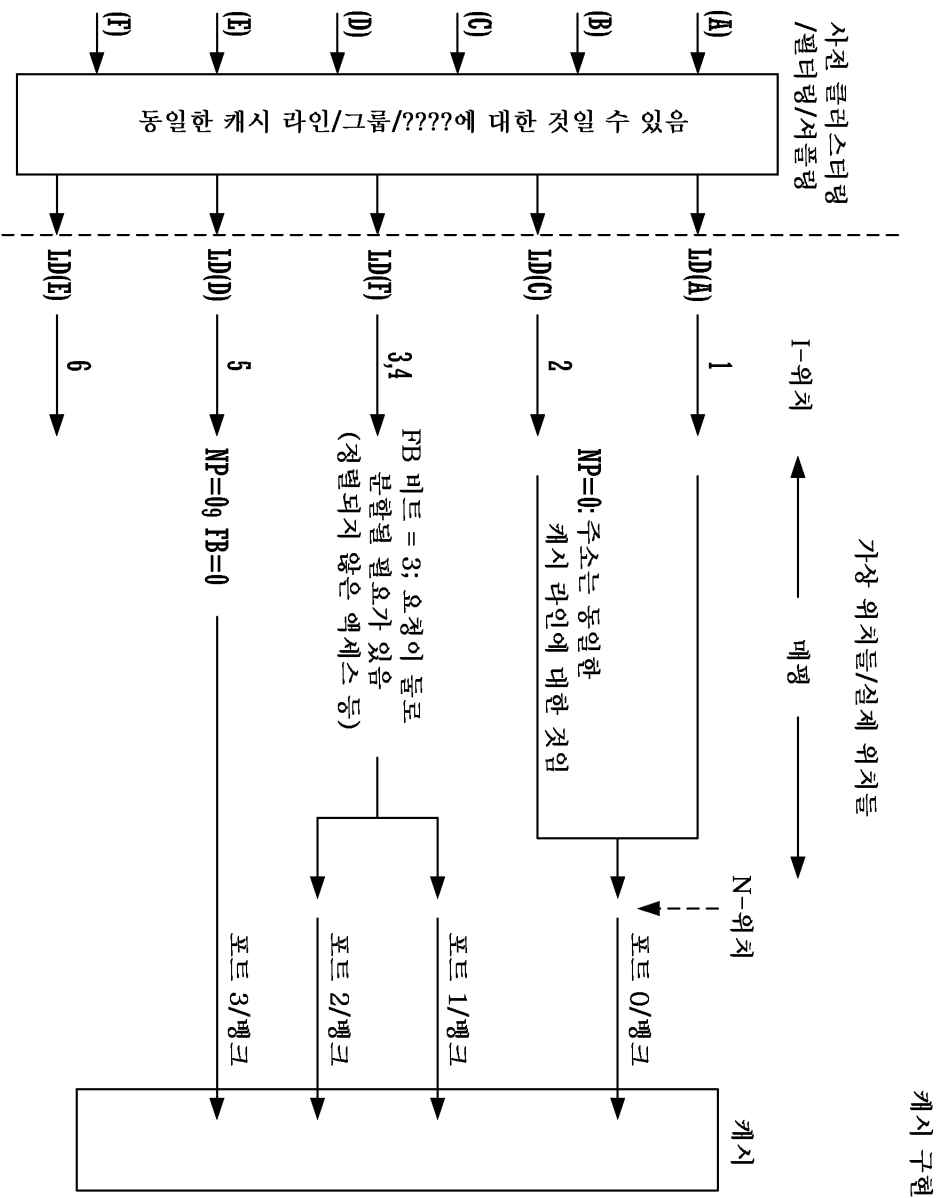


도면6

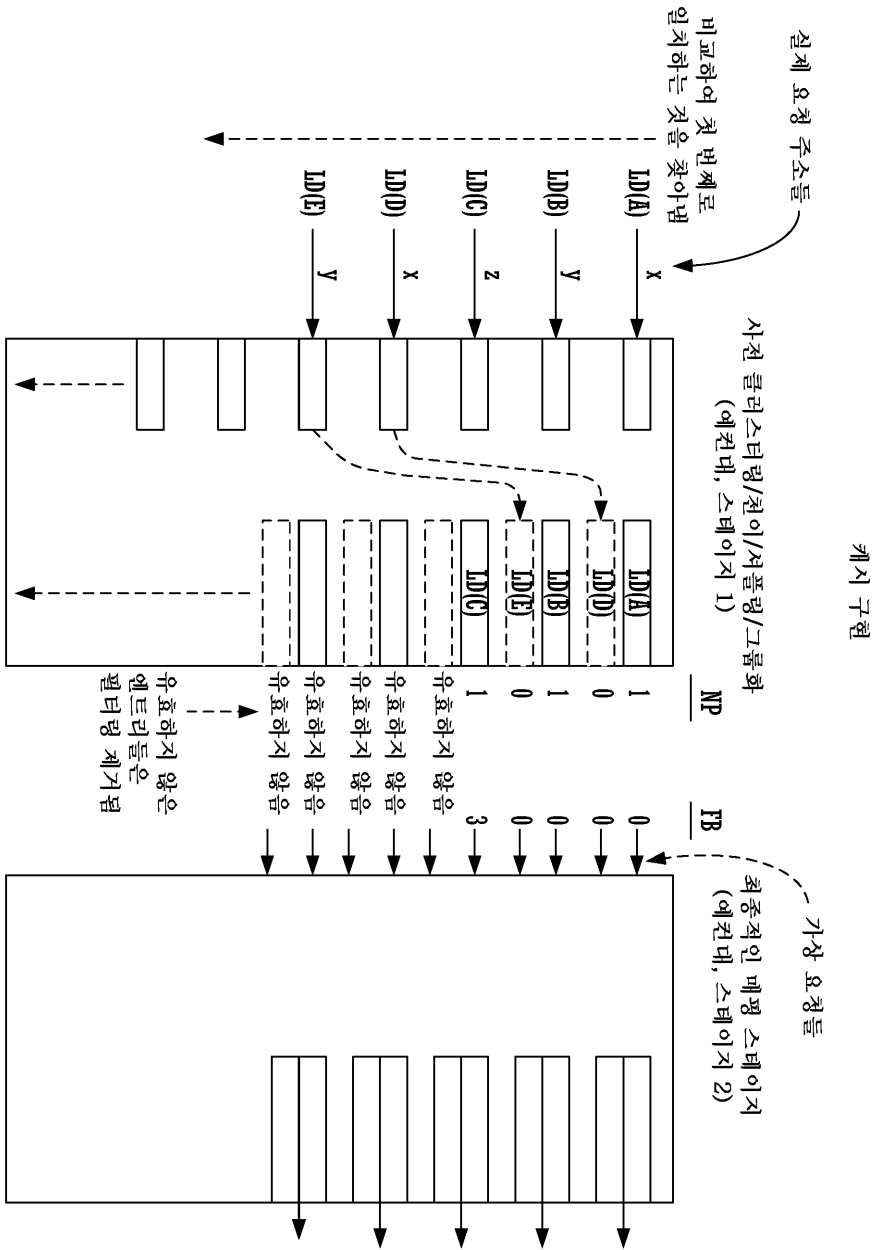
도면7



도면8

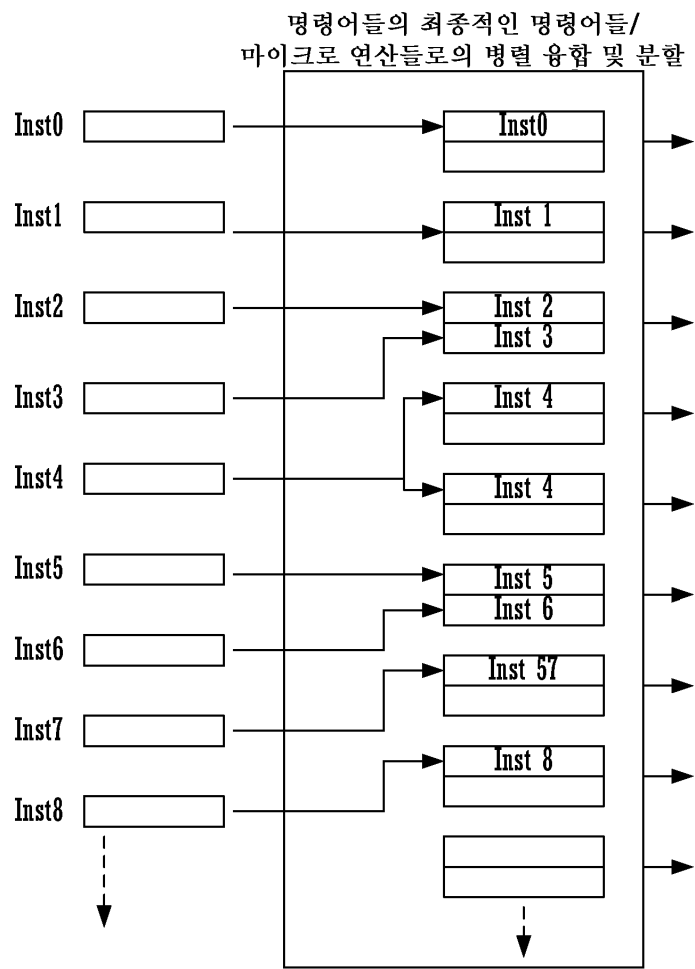


도면9



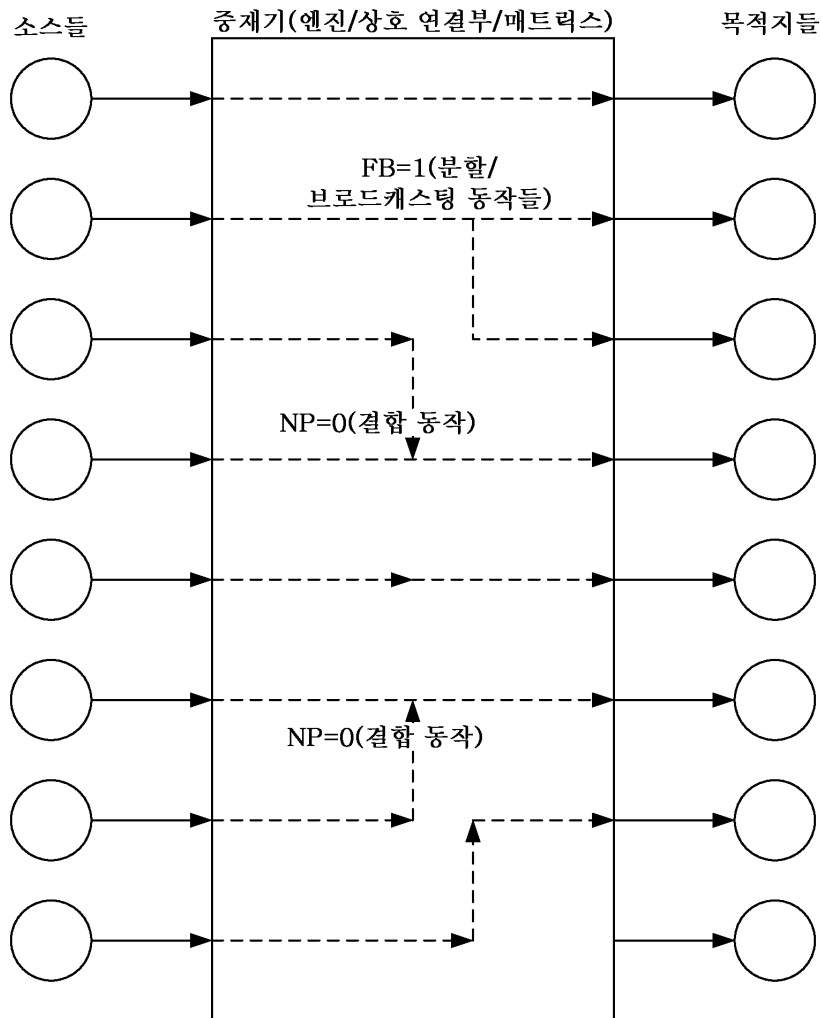
도면10

컴퓨터 명령어 구현



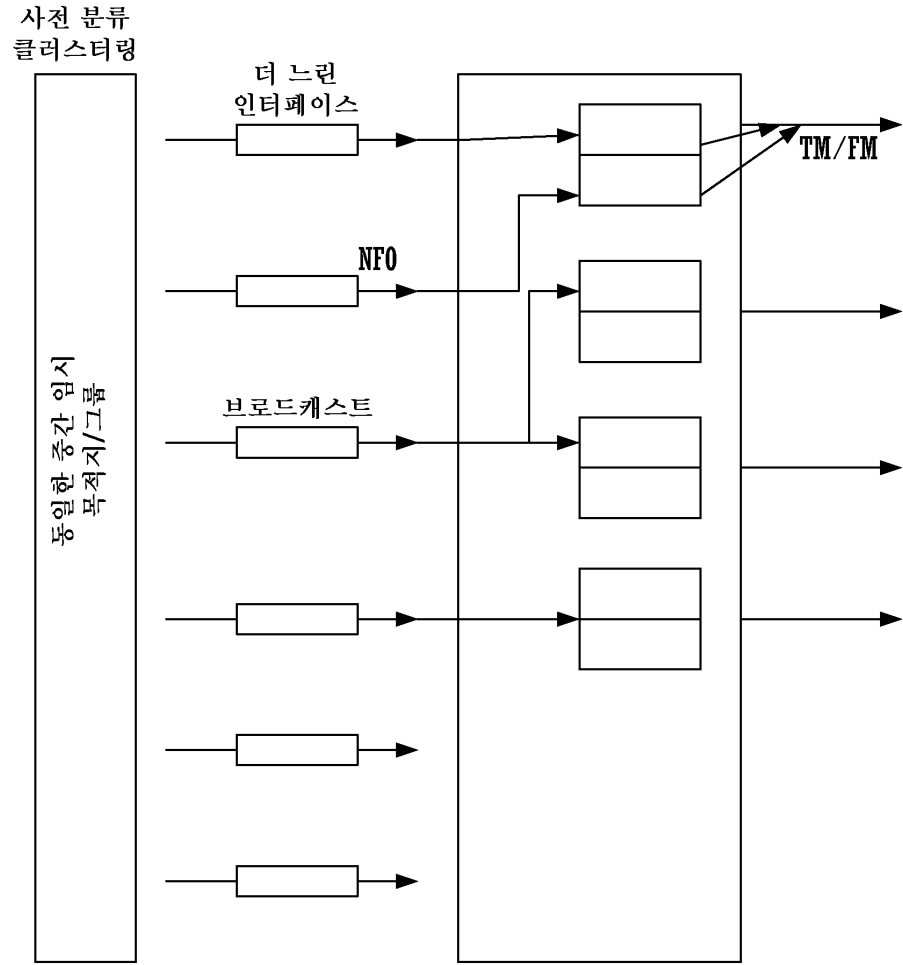
도면11

중재기 구현



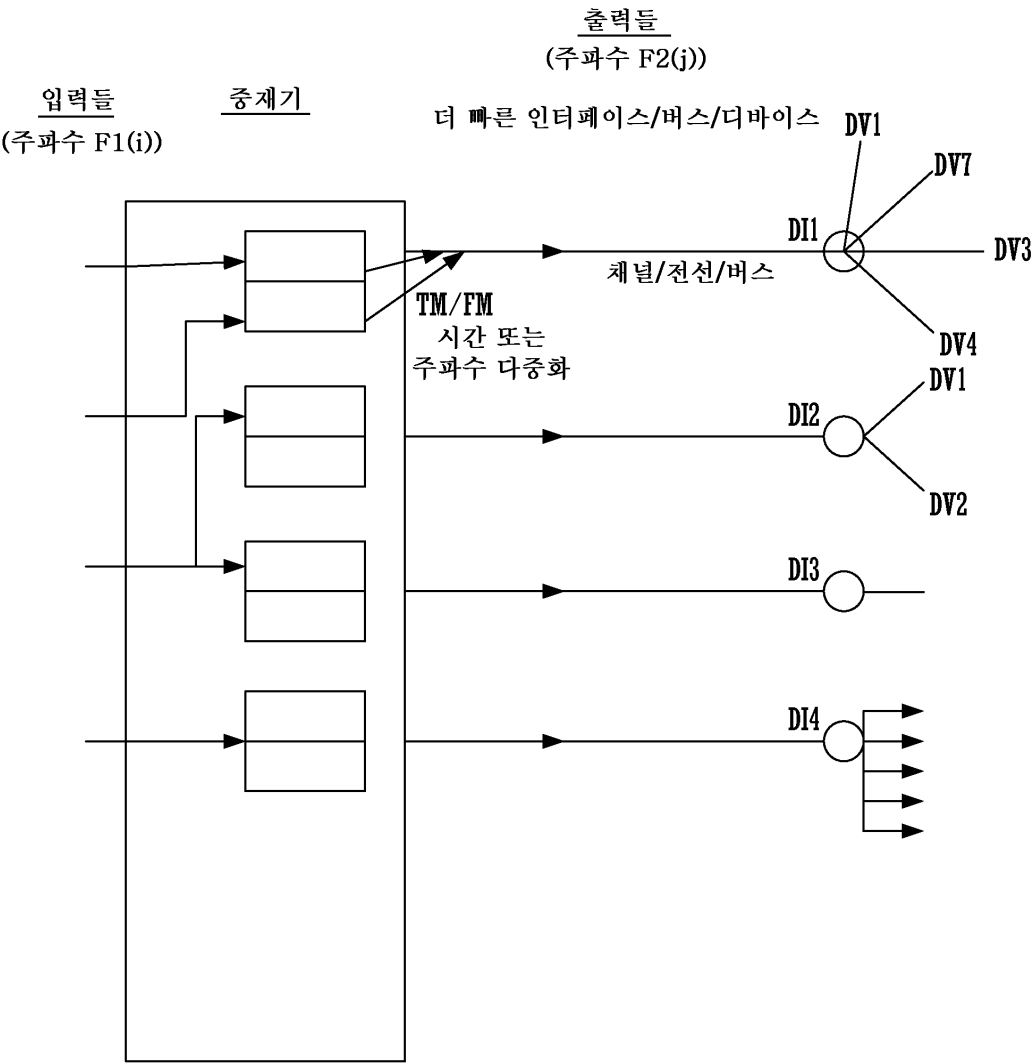
도면12

중재기 구현



도면13

중재기 구현

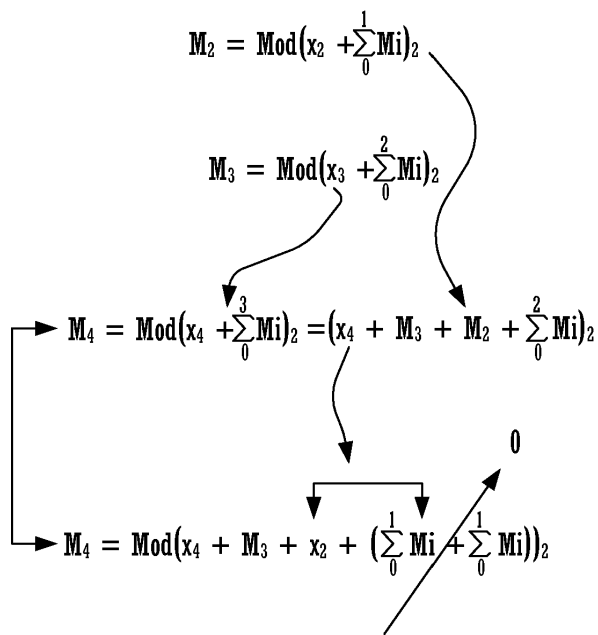


병렬 회선 속도 그룹화 및 분할

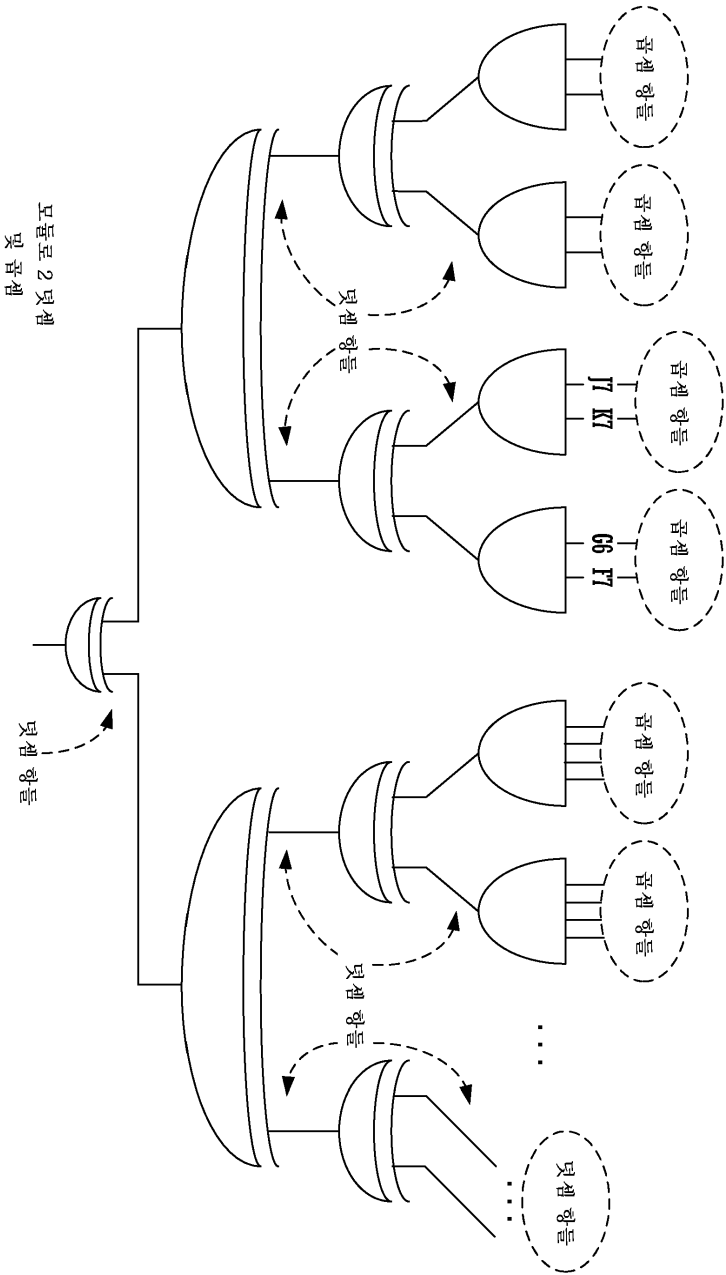
표인덱스	D	E	F	G	I	J	K	M	N			
명령어 시작 위치	초기 위치	(FB 또는 NP) [*]	FB NP	FB NP	FB NP	적렬 모드 FB/NP	병렬 모드 FB/NP	새로운 명령어 위치	병렬 명령어 위치+1	B#	적렬 모드 FB/NP [*] (채우적)	명령어 위치
3	1	0	1	0	0	0	0	0	1	0	E3+OR(MOD(G3,2),I3) [*] (MODE3+MOD(SUM(MS2,M2),2,2))	E3+SUM(MS3,M3)-63
4	1	1	1	0	0	0	1	0	1	2	G4+OR(MOD(G4,2),I4) [*] (MODE4+MOD(SUM(MS2,M3),2,2))	E4+SUM(MS3,M4)-64
5	1	2	1	0	0	0	0	0	2	3	G5+OR(MOD(G5,2),I5) [*] (MODE5+MOD(SUM(MS2,M4),2,2))	E5+SUM(MS3,M5)-65
6	1	3	0	0	1	1	1	1	4	5	G6+OR(MOD(G6,2),I6) [*] (MODE6+MOD(SUM(MS2,M5),2,2))	E6+SUM(MS3,M6)-66
7	1	4	0	1	1	1	0	2	6	7	G7+OR(MOD(G7,2),I7) [*] (MODE7+MOD(SUM(MS2,M6),2,2))	
8	1	5	1	0	0	0	1	0	8	9	G8+OR(MOD(G8,2),I8) [*] (MODE8+MOD(SUM(MS2,M7),2,2))	
9	1	6	0	3	0	1	0	4	10	11	G9+OR(MOD(G9,2),I9) [*] (MODE9+MOD(SUM(MS2,M8),2,2))	
10	1	7	0	0	1	1	1	0	14	15	G10+OR(MOD(G10,2),I10) [*] (MODE10+MOD(SUM(MS2,M9),2,2))	

표인덱스	D	E	F	G	I	J	K	M	N			
V-인덱스	명령어 시작	초기 위치	(FB 또는 NP)	FB NP	FB 또는 NP	모드 위치	적렬 모드 FB/NP	병렬 모드 FB/NP	새로운 명령어 위치	병렬 명령어 위치 + 1	B#	
3	1	0	1	0	0	0	0	0	0	1	0	$G3+I3^{*}MOD(K3,2)$
4	1	1	1	0	0	0	1	0	1	2	0	$G4+I4^{*}MOD(K4+G3+(I3^{*}K3),2)$
5	1	2	1	0	0	0	0	0	2	3	1	$G5+I5^{*}MOD(K5+G4+(I4^{*}K4)+(I4^{*}G3+(I4^{*}I3^{*}K3),2)$
6	1	3	0	0	1	1	1	1	4	5	2	$G6+I6^{*}MOD(K6+G5+(I5^{*}K5)+(I5^{*}G4+(I5^{*}I3^{*}K3),2)$
7	1	4	0	1	1	1	0	2	6	7	3	$G7+I7^{*}MOD(K7+G6+(I6^{*}K6)+(I6^{*}G5+(I6^{*}I3^{*}K3),2)$
8	1	5	1	0	0	0	1	0	8	9	4	$G8+I8^{*}MOD(K8+G7+(I7^{*}K7)+(I7^{*}G6+(I7^{*}I3^{*}K3),2)$
9	1	6	0	3	0	1	0	4	10	11	5	
10	1	7	0	0	1	1	1	0	14	15	7	
												병렬 명령어 위치
												$SUM(OS3,D3)+SUM(MS3,M3)-G3$
												$SUM(OS3,D4)+SUM(MS3,M4)-G4$
												$SUM(OS3,D5)+SUM(MS3,M5)-G5$
												$SUM(OS3,D6)+SUM(MS3,M6)-G6$

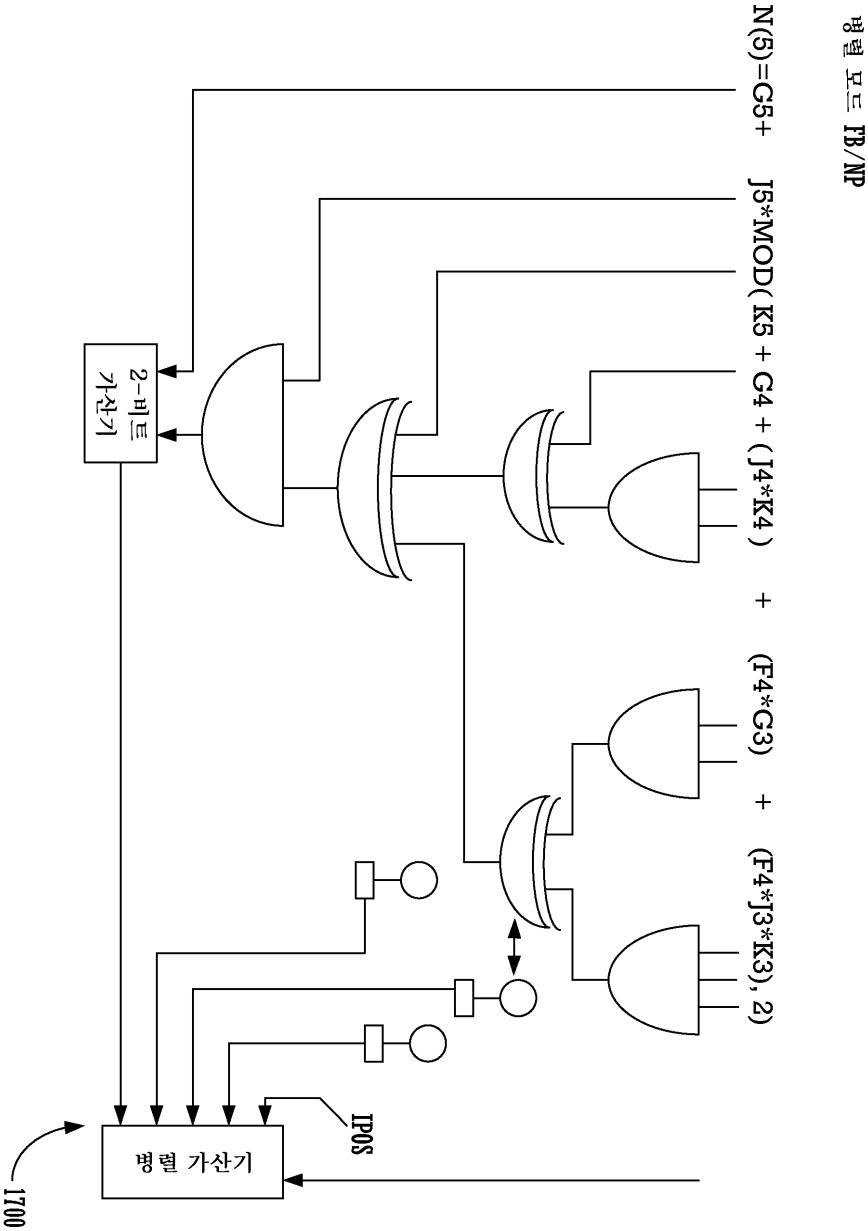
도면15



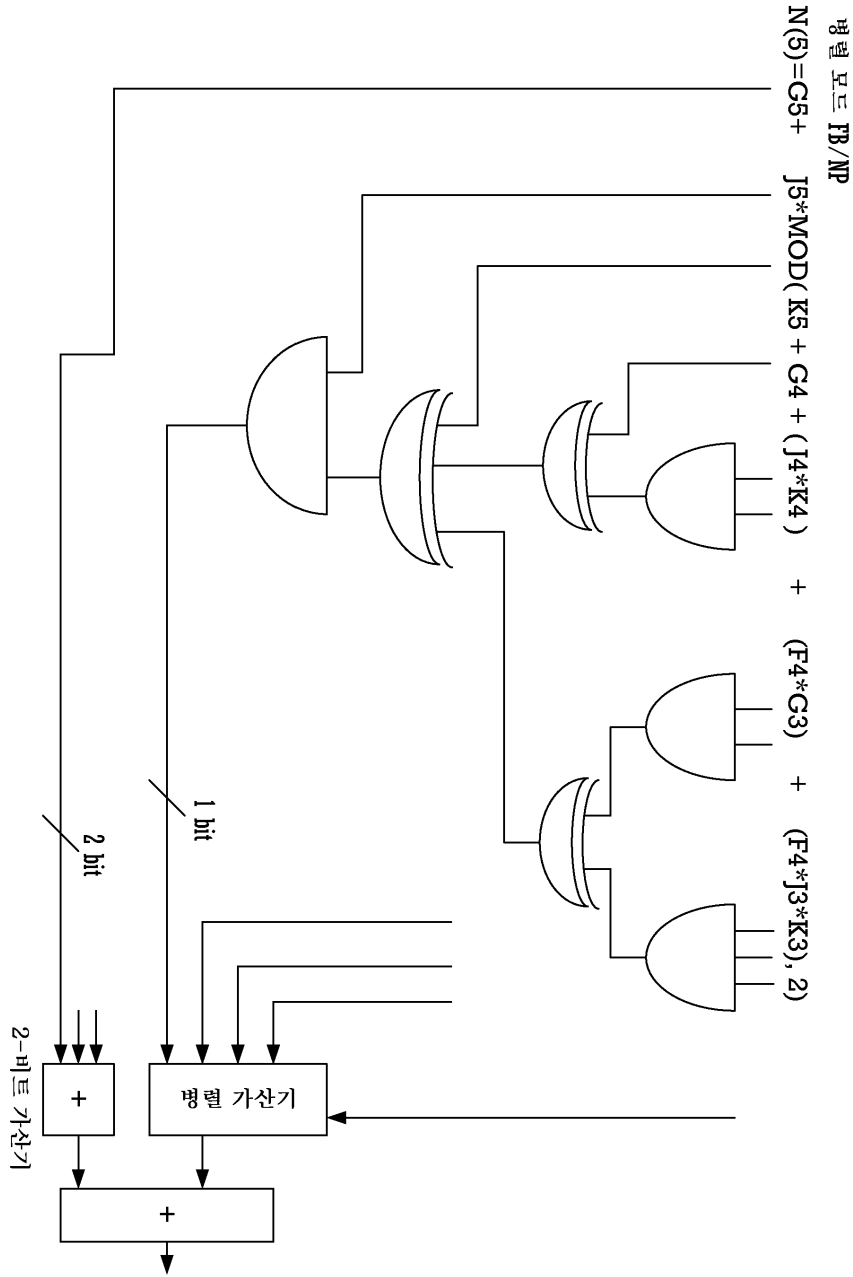
도면16



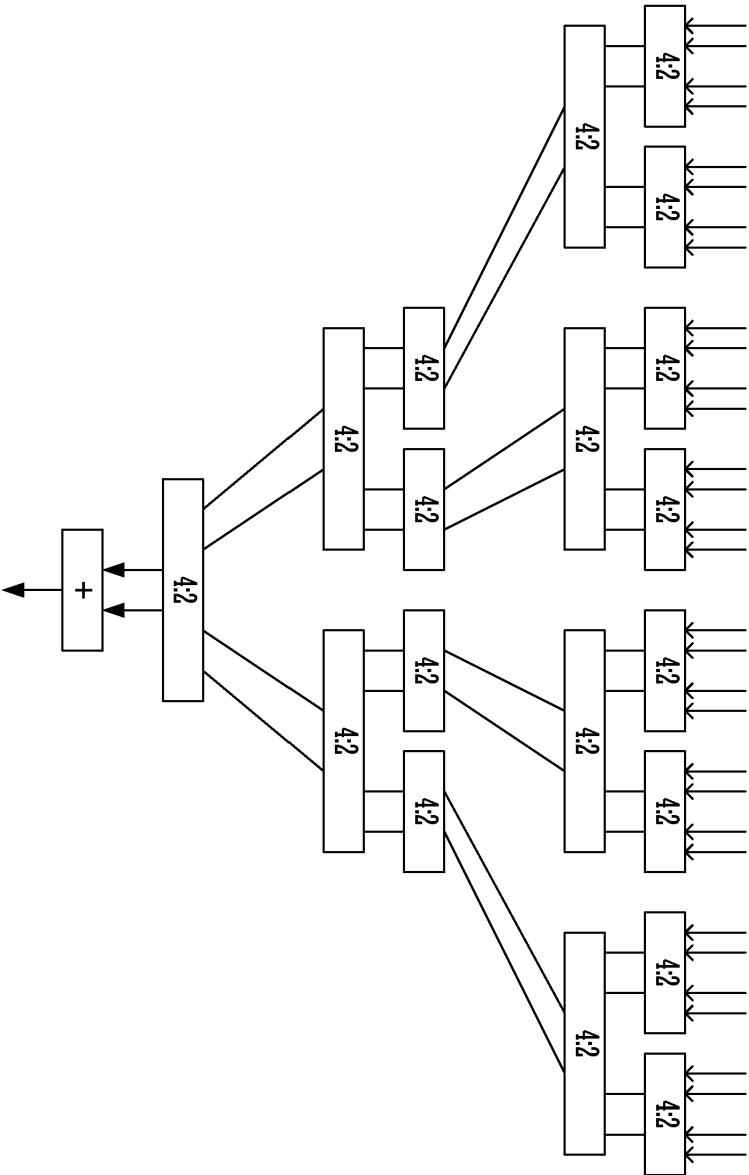
도면17



도면18

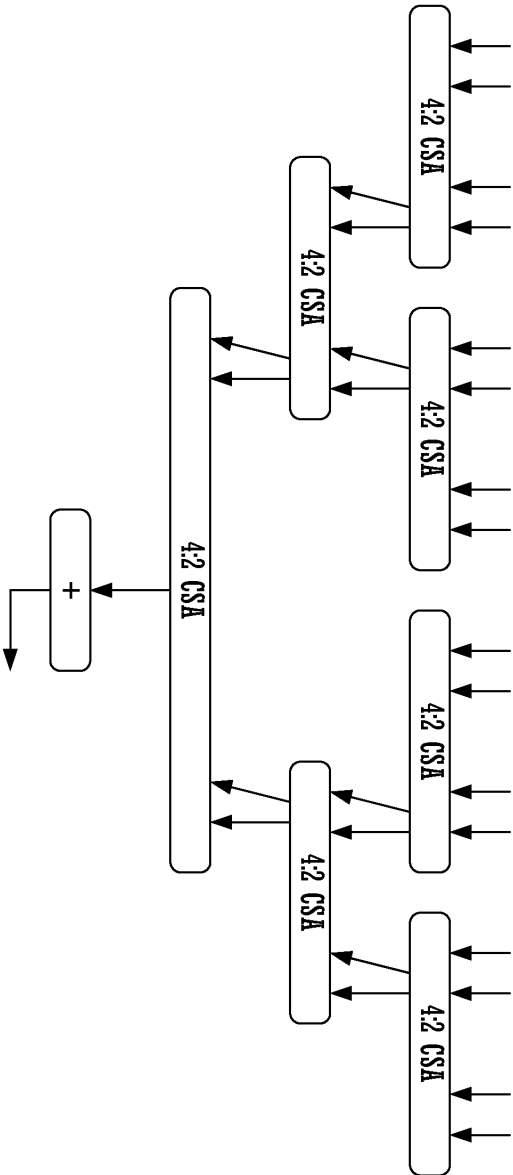


도면19

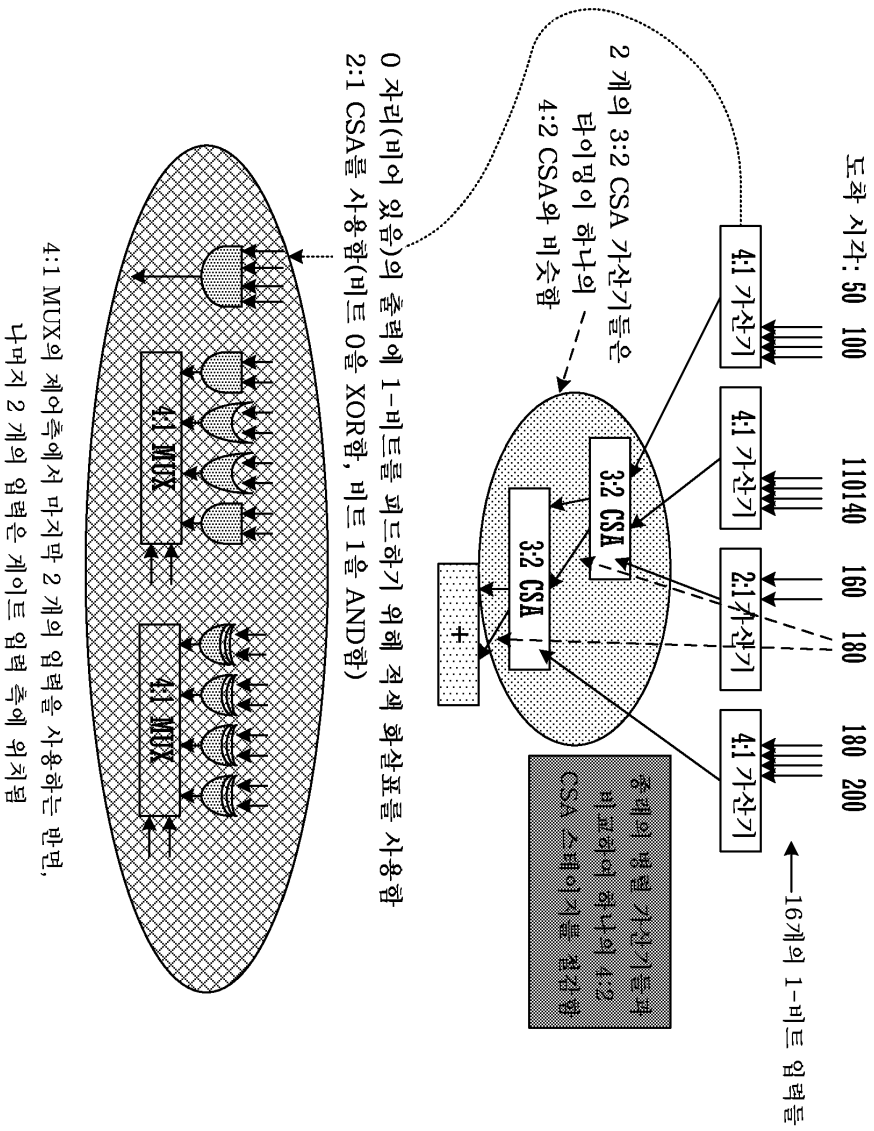


도 9로부터의 예시적인 가산기 (x)의 캐리 저장 가산기 병렬 구현

도면20



준비 원료/유효/회수 비트를 사용하는 1-비트 가산기에 의존하는 고속 병렬 가산기 구조들



도면21